

Ingegneria, Gestione ed Evoluzione del Software
Maintenance_Report
RAAF_GAMING



Francesco Peluso mat. 0522501771
Antonio Maddaloni mat. 0522501678
Antonio De Lucia mat. 0522501677

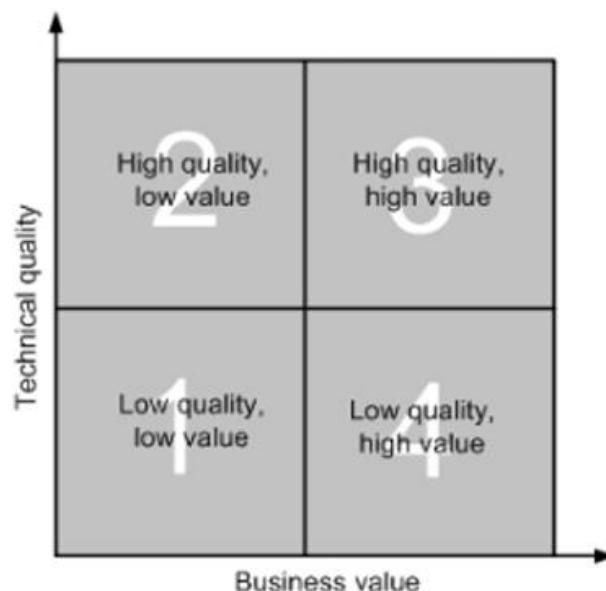
1. Introduzione e Business Case

1.1. Obiettivo del Documento

Il presente documento illustra l'iniziativa di reingegnerizzazione del sistema Raaf_Gaming, il cui obiettivo principale è la migrazione completa dalla piattaforma Java Legacy a un ambiente PHP/Laravel.

Poiché ogni processo di migrazione rappresenta anche un'opportunità di evoluzione, l'intervento non si limiterà al semplice trasferimento tecnologico: si prevede infatti di migliorare e ottimizzare l'architettura complessiva del sistema. Tali aspetti, insieme alle modifiche specifiche, alle scelte progettuali e agli interventi architetturali previsti, saranno descritti in dettaglio nei capitoli successivi del presente documento.

1.2. Portfolio Analysis



Posizionamento: Quadrante 4.

Motivazione Principale: Il sistema, pur presentando una struttura logica interna coerente e un'ingegnerizzazione del codice corretta, soffre di una marcata **obsolescenza tecnologica** dovuta all'assenza di un framework di supporto moderno.

Qualità Tecnica Basso: Nonostante l'ingegnerizzazione originale seguisse le best practices standard per lo sviluppo Java nativo (JSP/Servlet), la qualità tecnica attuale risulta insufficiente per sostenere l'evoluzione futura, a causa di obsolescenza tecnologica e limiti architetturali:

- **Obsolescenza:** L'assenza di un moderno framework di supporto ha costretto all'uso di pattern ormai superati (come ad esempio: DAO/DTO) rispetto agli attuali standard (come ad esempio: ORM), aumentando la verbosità e il costo di manutenzione.
- **Accoppiamento:** Si rileva un forte accoppiamento tra la logica di presentazione e quella di business, spesso incorporata direttamente nelle Servlet.
- **Debolezza Architetturale:** L'assenza di un Application Layer ha portato a una dispersione della logica di business.

Valore Commerciale Alto: Raaf_Gaming è l'applicazione centrale per il business dell'organizzazione, ecco perché si sta investendo in una sua completa reingegnerizzazione.

2. Analisi Del Sistema Attuale

2.1. Descrizione del Progetto e Ambito Applicativo

Il progetto consiste in una piattaforma web di e-commerce, specializzata nel mercato videoludico. Il sistema propone acquisti di prodotti rispetto ad un catalogo disponibile e aggiornabile, che include **videogiochi** (sia fisici che digitali/DLC), **console** e **abbonamenti** ai servizi di gaming.

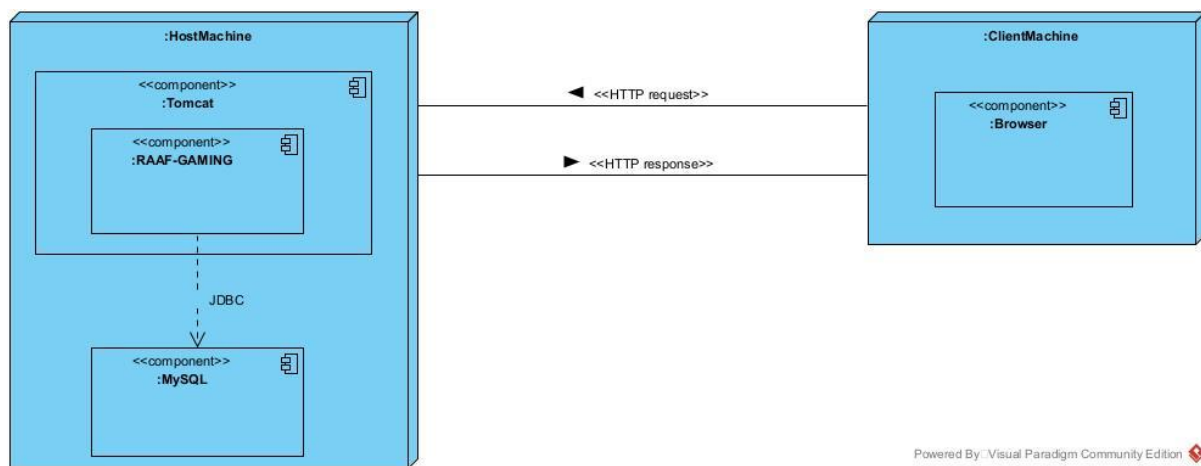
Oltre alle funzionalità core di vendita online, il sistema è progettato per favorire la creazione di **feedback**, permettendo lo scambio di opinioni tramite un sistema di recensioni sui prodotti.

Il sistema gestisce l'intero ciclo di vita dell'ordine, offrendo funzionalità distinte per quattro tipologie di attori:

1. **Clienti (Visitatori e Registrati):** Navigazione catalogo, gestione carrello, acquisto, tracking ordini e gestione profilo/carta fedeltà.
2. **Gestori Backoffice (Ordini e Magazzino):** Gestione logistica, approvvigionamento stock (Restock), inserimento nuovi prodotti e gestione delle spedizioni.

2.2. Architetture e Componenti

La documentazione esistente e l'analisi del codice sorgente si sono rivelati sufficienti per comprendere appieno l'intero sistema e le sue funzionalità. Di conseguenza, non è stato necessario ricorrere a strumenti o procedure complesse di reverse engineering per ricostruire la logica del sistema, che è risultata chiara fin da subito.



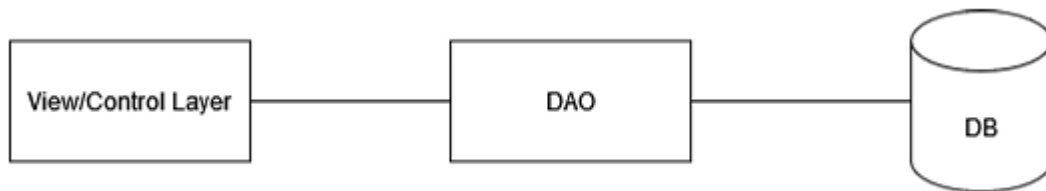
Il diagramma di deployment in Figura illustra l'architettura fisica del sistema. L'infrastruttura segue il modello **Client-Server** tradizionale, con una centralizzazione delle risorse di calcolo e di storage su un unico nodo server.

Gli elementi costitutivi sono:

- **Client Machine:** Rappresenta il dispositivo dell'utente finale. L'interazione avviene esclusivamente tramite il componente **Browser**. Non vi è logica applicativa installata sul client; esso si limita a inviare richieste ed a indirizzare le risposte HTML.
- **Host Machine (Server):** È il nodo centrale che ospita l'intero stack applicativo. Al suo interno risiedono:
 - **Apache Tomcat:** Il Web Server e Servlet Container che gestisce il ciclo di vita dell'applicazione e le connessioni HTTP.
 - **RAAF-GAMING:** L'artefatto software (applicazione web J2EE) distribuito all'interno del container Tomcat.
 - **MySQL:** Il DBMS relazionale che risiede sulla stessa macchina fisica dell'applicazione.

Protocolli di Comunicazione:

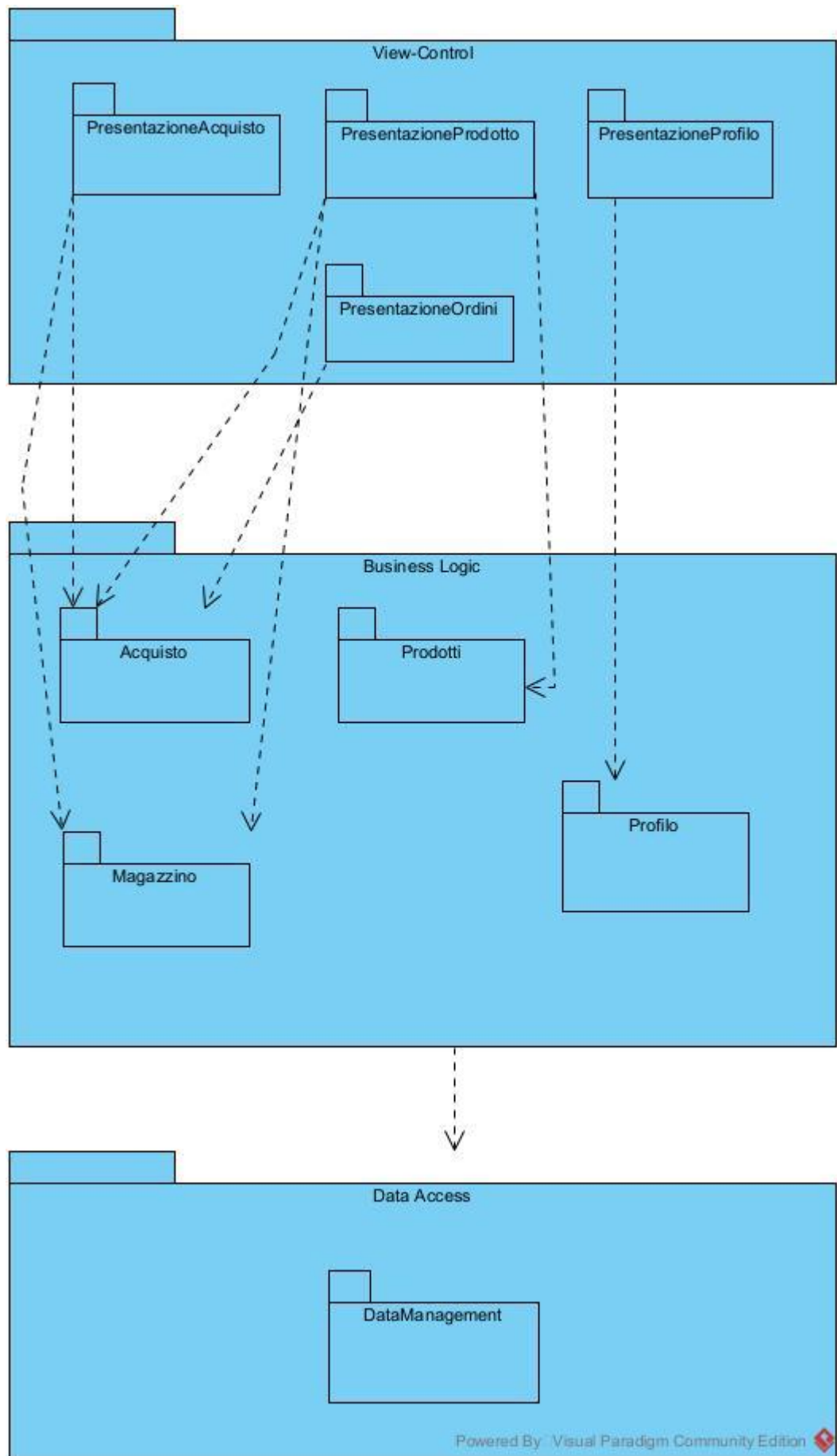
- **HTTP (Request/Response):** Canale di comunicazione standard tra il Client (Browser) e il Server (Tomcat).
- **JDBC (Java Database Connectivity):** Protocollo utilizzato dal componente *RAAF-GAMING* per comunicare direttamente con il database *MySQL*.



La Figura illustra l'architettura logica del componente *RAAF-GAMING*, organizzata secondo un pattern **Three-Tier**.

L'architettura è suddivisa nei seguenti livelli funzionali:

1. **View/Control Layer (Livello di Presentazione):**
In questo livello risiedono sia la logica di presentazione (JSP) che la logica di controllo (Servlet). Nel sistema Legacy, questi due aspetti non sono nettamente separati: le componenti di visualizzazione spesso interagiscono direttamente con la logica di business o invocano il recupero dei dati.
2. **DAO Layer:** Rappresenta il livello di astrazione per l'accesso ai dati. Questo layer isola la business logic dalla complessità delle query SQL. Il View/Control Layer invoca i metodi del DAO per ottenere oggetti Java (Bean) popolati con i dati grezzi.
3. **DB (Database):** Rappresenta il livello di persistenza fisica (MySQL), dove risiedono i dati relazionali.



Come illustrato nel diagramma, il sistema è strutturato logicamente in tre macro-aree che raggruppano le funzionalità principali. Questa suddivisione definisce le Componenti che saranno oggetto della migrazione:

1. Business Logic: Questa sezione racchiude i moduli funzionali principali:

- **Acquisto:** Gestisce le logiche del carrello e la finalizzazione dell'ordine.
- **Prodotti:** Gestisce il catalogo, la ricerca e le informazioni di dettaglio dei videogiochi/console.
- **Magazzino:** Gestisce le operazioni di backoffice per il rifornimento stock e nuovi inserimenti.
- **Profilo:** Gestisce i dati dell'utente, l'autenticazione e la carta fedeltà.

2. View-Control: Per ogni modulo di business esiste una corrispettiva componente di presentazione che gestisce l'interazione con l'utente:

- **PresentazioneAcquisto** (interfaccia carrello/checkout).
- **PresentazioneProdotto** (interfaccia catalogo).
- **PresentazioneProfilo** (interfaccia utente/login).
- **PresentazioneOrdini** (interfaccia storico e gestione spedizioni).

3. Data Access

- **DataManagement:** È il componente trasversale che centralizza l'accesso al database per tutte le funzionalità sopra descritte.

3. Impact Analysis

3.1. Change Request

La presente Change Request ha come obiettivo principale la **migrazione** del sistema, attualmente operante su una piattaforma basata su **tecnologia Java (JSP/Servlet)**. La scelta di intraprendere questa operazione è dettata dalla necessità di affrontare l'obsolescenza tecnologica accumulata e di garantire l'efficienza e la manutenibilità a lungo termine della piattaforma. Il cuore della CR è la **migrazione completa** dell'intero stack applicativo verso un ambiente moderno, identificato nello **stack PHP/Laravel**. Tale operazione comporta la sostituzione dei seguenti layer applicativi:

- **Backend.**
- **Frontend.**

La migrazione non si limiterà a replicare le funzionalità esistenti nel nuovo linguaggio, ma sarà utilizzata per **migliorare e ottimizzare l'architettura** complessiva del sistema. Questo approccio garantisce che la nuova implementazione in **PHP/Laravel** sia non solo funzionalmente equivalente, ma anche **superiore strutturalmente**, più manutenibile rispetto alla versione Legacy, l'unica componente del sistema che sarà **totalmente mantenuta e riutilizzata** è la **base dati esistente**. Ciò assicura la continuità storica e l'integrità dei dati operativi, non avendo quasi alcun impatto sui processi di *data migration* e permettendo al team di focalizzarsi interamente sulla riscrittura e ottimizzazione del software applicativo. Infine è importante chiarire che, sostituendo l'attuale implementazione Java con PHP/Laravel, **il codice non potrà naturalmente essere riutilizzato**. Verrà però preservata la logica di fondo laddove ha ancora senso mantenerla, mentre saranno introdotti miglioramenti architetturali e funzionali nelle parti in cui l'attuale struttura risulta limitante. Ad esempio, sarà possibile estrarre alcuni controlli oggi presenti nelle servlet e riorganizzarli in

middleware dedicati (come quelli per l'autenticazione), ottenendo una separazione più pulita delle responsabilità. Per il resto, la logica core rimarrà concettualmente simile, salvo gli interventi necessari a modernizzare e rafforzare la business logic.

3.2. Start Impact Set (SIS)

Sulla base della Change Request, lo **Start Impact Set** comprende tutti gli artefatti del sistema che necessitano di modifica o riscrittura. Dato l'obiettivo di migrazione completa verso un nuovo stack tecnologico, il SIS include:

1. Codice Sorgente:

- L'intero contenuto dei package Java (**View-Control, DAO, Data Access**), rispettivo per i package di test, che saranno oggetto di traduzione e porting.

2. System Design:

- L'architettura software attuale (J2EE/Servlet-based) viene invalidata. Rientrano nel SIS il diagramma di **Hardware & Software Mapping**, che dovrà essere aggiornato per riflettere la nuova infrastruttura.

3. Object Design:

- I diagrammi riferito al punto di **Packages and Class Interfaces**, rientrano nel SIS. Sebbene le entità concettuali rimangano le stesse, la loro struttura cambia radicalmente (es. una Servlet viene definita diversamente dai Controller Laravel, anche se logicamente sono simili).

Nota sugli Artefatti Esclusi:

Restano invece esclusi dal SIS gli artefatti di alto livello come il RAD, dove i Casi d'Uso e i Diagrammi di Sequenza non cambiano, poiché la nostra change request non aggiunge requisiti o funzionalità, ma impatta solo la parte di design dell'intero sistema.

$$\text{SIS} = \{\text{Codice Sorgente}\} \cup \{\text{Artefatti-System-Design}\} \cup \{\text{Artefatti-Object-Design}\}$$

3.3. Candidate Impact Set (CIS) e Ripple Effect

Data la natura di **migrazione massiva** della CR, l'analisi di impatto si configura come segue:

1. **Ripple Effect:** È da considerarsi **nullo/irrilevante**. Non si verifica propagazione dell'errore su moduli non toccati, poiché **l'intero sistema applicativo è oggetto di riscrittura**.
2. **CIS:** Per coerenza, esso **coincide con l'intero sistema applicativo target**. Tutti i moduli, parti del system e object design che verranno implementati sono inclusi.

4. Strategia Di Migrazione

4.1. Approccio Metodologico

Trattandosi di una **migrazione massiva e completa** del sistema, non è applicabile una metodologia incrementale “pura” in senso tradizionale, poiché l'obiettivo non è introdurre nuove funzionalità in modo progressivo, ma trasferire integralmente l'esistente su una nuova base tecnologica. Di conseguenza, il progetto non segue una metodologia rigida di migrazione modulare con rilasci intermedi funzionalmente autonomi, in quanto il sistema

deve essere migrato nella sua interezza prima di poter essere considerato pienamente operativo. Tuttavia, per evitare un approccio destrutturato e garantire coerenza, tracciabilità e controllo dell'avanzamento, è stata adottata una **Strategia Ibrida** che fornisce una linea guida metodologica generale senza imporre vincoli incompatibili con la natura della migrazione:

1. **Integrazione:** Pur non adottando una metodologia incrementale “pura”, la riscrittura del codice viene organizzata secondo una logica progressiva per componenti funzionali. Il sistema è stato scomposto in moduli, non ai fini di rilasci intermedi autonomi, ma per strutturare e governare il lavoro di migrazione. Per ciascun modulo viene completato l'intero stack operativo (Design -> Implementazione -> Test di Unità), garantendo controllo e qualità a livello di singolo componente, pur rimandando la validazione sistemica al completamento dell'intera migrazione.
2. **Rilascio: Metodo Cold Turkey** - Data la natura radicale del cambiamento tecnologico (da Java a PHP) e l'impossibilità tecnica di condividere lo stato della sessione utente tra due application server eterogenei senza complessi meccanismi di bridge, non è prevista una fase di coesistenza in produzione. Il nuovo sistema sostituirà integralmente il Sistema Legacy in un unico evento di cut-over, una volta completata la migrazione di tutti i componenti.

4.2. Piano di Lavoro

Fase	Componente	Dipendenze	Descrizione dell'Intervento
Fase 1	Hardware & Software Mapping (SDD ONLY)	Nessuna	Bisogna modificare quello esistente con il nuovo
Fase 2	Utility	Fase 1	Bisogna progettare e implementare il modulo utility
Fase 3	Profilo	Fase 2	Bisogna progettare e implementare il modulo profilo più i relativi test di unità
Fase 4	Magazzino	Fase 3	Bisogna progettare e implementare il modulo magazzino più i relativi test di unità
Fase 5	Prodotto	Fase 4	Bisogna progettare e implementare il modulo di prodotto con i

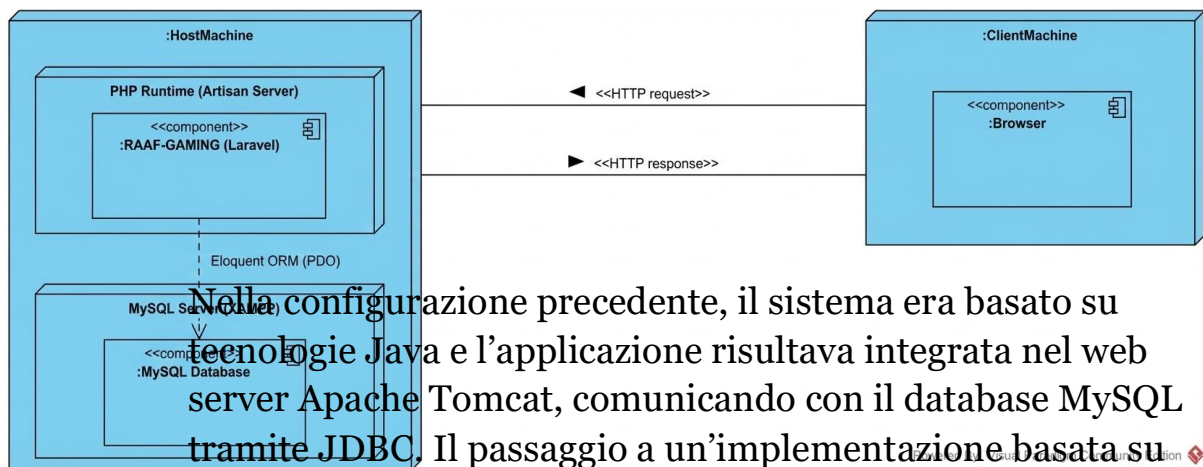
			relativi test di unità
Fase 6	Acquisto	Fase 5	Bisogna progettare e implementare il modulo di acquisto con i relativi test di unità
Fase 7	Test Di Sistema	Fase 6	Bisogna progettare e implementare i moduli dei test di sistema

5. Implementazione

5.1. Fase 1

Per completare la Fase 1 della migrazione, precedentemente preventivata e schedata durante la pianificazione delle attività di manutenzione evolutiva, si è reso necessario intervenire sull'hardware & software mapping del sistema. Tale modifica rappresenta il primo intervento architetturale da effettuare, in quanto strettamente legato al cambio di architettura di deployment e al passaggio tecnologico da Java a PHP, coerentemente con le tecnologie effettivamente utilizzate nel nuovo stack applicativo.

Hardware & Software Mapping Nuovo:



Nella configurazione precedente, il sistema era basato su tecnologie Java e l'applicazione risultava integrata nel web server Apache Tomcat, comunicando con il database MySQL tramite JDBC. Il passaggio a un'implementazione basata su PHP e Laravel ha reso questa configurazione non più adeguata, rendendo necessaria una ridefinizione della mappatura tra componenti software e risorse hardware.

Come mostrato nel nuovo hardware & software mapping:

- La ClientMachine deve ospitare un Browser, che interagisce con il sistema tramite richieste e risposte HTTP (Uguale a com'era stato definito prima della modifica).
- L'HostMachine ospita invece, la componente PHP Runtime (Artisan Server), che è quel web server built-in integrato direttamente nel runtime di PHP, all'interno del quale è deployata l'applicazione RAAF-GAMING, ora implementata come componente Laravel.
- L'accesso ai dati persistenti avviene tramite Eloquent ORM, che utilizza PDO (l'equivalente JDBC per PHP ma automatizzato) per comunicare con il MySQL Server, eseguito a sua volta in ambiente XAMPP.
- Il MySQL Database risulta separato dal runtime applicativo e mappato come componente indipendente

sullo stesso host.

Questa modifica riflette esclusivamente il cambiamento delle tecnologie di esecuzione e di persistenza, senza introdurre variazioni funzionali al sistema. Infine la ridefinizione dell'hardware–software mapping costituisce l'unica modifica al System Design necessaria in questa fase della migrazione.

La struttura dei pacchetti, i componenti logici e le altre viste architetturali del sistema rimangono invariate, in quanto il cambiamento riguarda esclusivamente il deployment e lo stack tecnologico adottato.

5.2. Fase 2

In questa fase, è stata completata l'inizializzazione del nuovo progetto. Sono stati installati e configurati gli strumenti fondamentali per lo sviluppo dell'applicazione, senza intervenire sulla documentazione esistente, ma operando direttamente sul codice e sulla struttura del progetto.

In particolare, sono state utilizzate le seguenti tecnologie:

- **XAMPP 3.3.0**, come ambiente di sviluppo locale;
- **PHP 8.2.x**, come linguaggio di backend;
- **Composer 2.8.9**, come gestore delle dipendenze PHP.

Grazie a queste tecnologie è stato possibile inizializzare un progetto standard Laravel, installando preventivamente l'installer globale di Laravel tramite il comando:

```
composer global require laravel/installer
```

Successivamente, è stato creato lo scheletro del progetto mediante il comando:

```
laravel new RAAF-GAMING-NEW
```

Una volta generata la struttura base di Laravel, si è proceduto alla configurazione iniziale del progetto. In questa fase sono stati importati e riorganizzati alcuni file già esistenti e destinati al riutilizzo, come ad esempio le immagini dell'applicazione. Tali risorse, precedentemente collocate nella directory **WebContent/immagini**, sono state migrate nella cartella **public/immagini**, in accordo con la convenzione di Laravel per la gestione delle risorse pubbliche.

Alcuni file e directory presenti nel progetto originale, quali:

- **.project**
- **WEB-INF/web.xml**
- **WEB-INF/lib** (contenente le librerie in formato JAR)
- l'intera directory **META-INF**
- **src/it/unisa/utility/MainContext.java**

non sono stati trasferiti direttamente nel nuovo progetto Laravel, in quanto appartengono a una struttura e a un ecosistema tecnologico differenti.

Nel contesto Laravel, tali componenti vengono gestiti in modo diverso. Ad esempio, le librerie non sono incluse manualmente all'interno del progetto, ma vengono installate automaticamente tramite strumenti di dependency management come **Composer** o **NPM**, utilizzando i file di configurazione **composer.json** e **package.json**. Le directory **vendor** e **node_modules**, generate a seguito dei comandi **composer install** o **npm install**, non vengono modificate direttamente dallo sviluppatore.

Di conseguenza, non è necessario stabilire una corrispondenza uno-a-uno tra tutte le cartelle e i file del progetto precedente e quelli del nuovo progetto, poiché cambia radicalmente la logica di utilizzo e gestione delle risorse.

Un ulteriore esempio di questa differenza riguarda il file **MainContext.java**, precedentemente utilizzato per gestire la connessione al database. In Laravel, tale responsabilità è completamente demandata al framework, che fornisce un sistema di configurazione centralizzato e automatizzato.

La connessione al database viene configurata tramite il file **.env**, il quale contiene le variabili d'ambiente necessarie (host, nome del database, username, password). Queste variabili sono poi utilizzate dai file di configurazione presenti nella directory **config**. In questo modo, la modifica del file **.env** consente di aggiornare le configurazioni dell'applicazione senza intervenire direttamente sul codice sorgente. Pertanto, lo sviluppatore non deve preoccuparsi dell'implementazione interna della connessione al database, ma esclusivamente della corretta configurazione dei parametri richiesti da Laravel.

Nella fase iniziale della migrazione sono stati portati nel nuovo progetto i principali componenti di layout dell'applicazione, ovvero **navbar.jsp** e **footer.jsp**.

Per organizzare correttamente le viste secondo le convenzioni di Laravel, è stata innanzitutto creata la directory **resources/views/layouts**, destinata a contenere i template condivisi dell'applicazione. All'interno di questa cartella sono state definite tre viste Blade:

- **app.blade.php**, file di nuova creazione e non presente nel sistema legacy;
- **navigation.blade.php**, derivata dalla conversione di **navbar.jsp**;
- **footer.blade.php**, derivata dalla conversione di **footer.jsp**.

Il file **app.blade.php** svolge il ruolo di layout principale dell'applicazione. Esso è stato introdotto per garantire una migliore suddivisione del codice e per centralizzare l'inclusione dei componenti comuni, come la barra di navigazione e il footer, in tutte le viste future. In questo modo, anche le successive importazioni e migrazioni di viste verranno gestite tramite il layout principale, mantenendo il codice più ordinato e facilitando la manutenzione. Tale approccio consente inoltre di rispettare le linee guida di Laravel per l'utilizzo delle Blade templates.

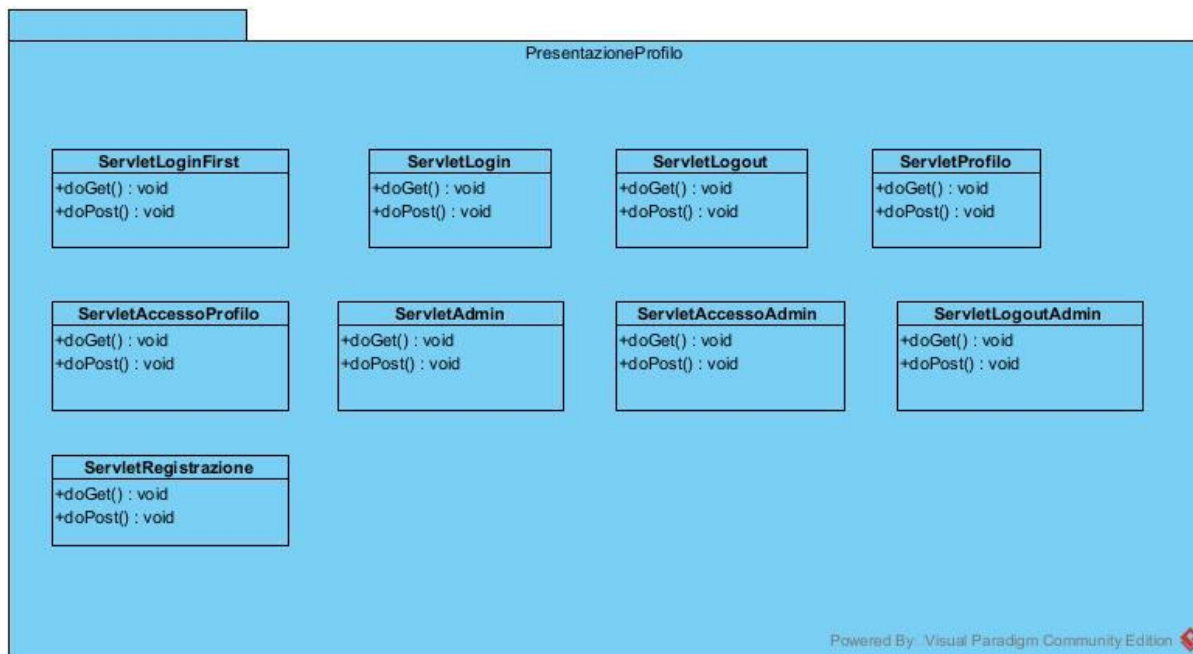
Per quanto riguarda la gestione delle risorse statiche, dopo la creazione delle viste Blade per navbar e footer, sono state aggiunte nella cartella **public** due nuove directory:

- **public/css**, contenente i file CSS relativi a navbar e footer, in continuità con la struttura presente nella cartella **WebContent/css** del progetto legacy;
- **public/js**, contenente i file JavaScript corrispondenti, che mappano la precedente cartella **WebContent/js** e che verranno utilizzati nelle fasi successive della migrazione.

Questa organizzazione consente una gestione più coerente delle risorse statiche e facilita l'estensione futura dell'applicazione all'interno del nuovo framework Laravel.

5.3. Fase 3

5.3.1. Package PresentazioneProfilo

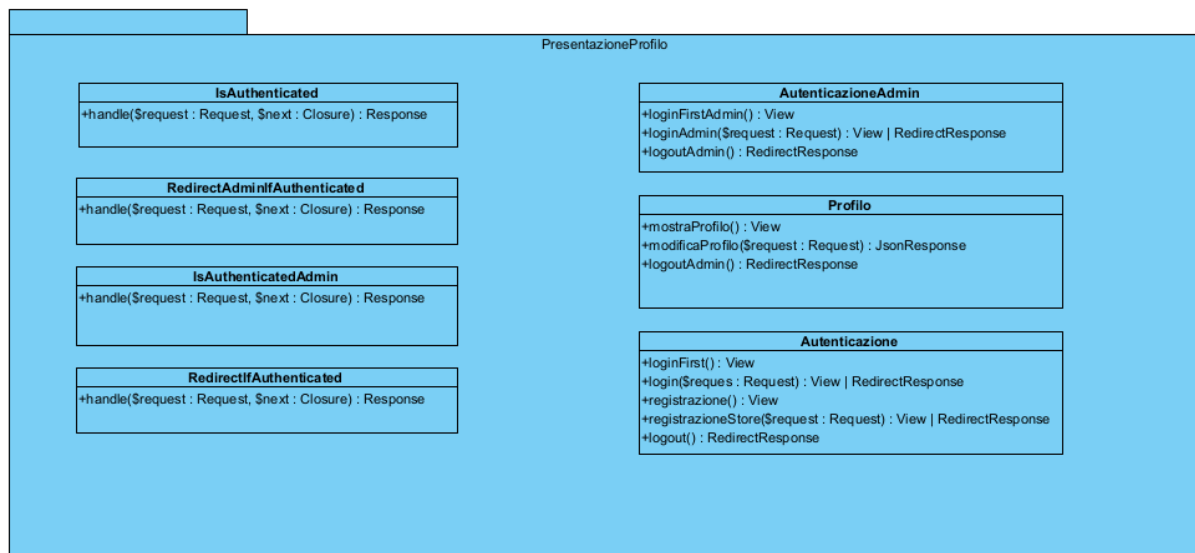


In questa fase della migrazione siamo partiti con la migrazione del package “PresentazioneProfilo”, su cui è stata effettuata un’analisi approfondita dei componenti esistenti nel sistema legacy, costituiti dalle rispettive pagine JSP e Servlet (**definite in figura**), questo perché definisce quello che noi chiamiamo layer o logica di presentazione. Questa analisi ha consentito di individuare in modo chiaro le responsabilità di ciascun componente e i diversi flussi, come quelli di autenticazione e registrazione (sia per utenti che per amministratori). Ciò ha reso possibile una loro riprogettazione all’interno di Laravel, nel rispetto di un paradigma architetturale più adeguato alla separazione della business logic (oltre ad una semplice traduzione).

Dall’analisi è infatti emerso che i controller accentravano gran parte della business logic senza un’adeguata suddivisione, facendo presupporre l’assenza o una presenza minimale del relativo package “Profilo”, che avrebbe invece dovuto raccogliere la logica di dominio e quella di model.

Dopo la fase di analisi, e parallelamente all'implementazione utile anche per comprendere meglio le scelte durante la fase di traduzione è emersa con maggiore chiarezza la necessità di riorganizzare pacchetti, classi e responsabilità. Infatti, nel corso della semplice traduzione da Java a Laravel, si sono evidenziate le reali esigenze dell'applicazione e il modo corretto di raccogliere tali informazioni all'interno di un layer di business logic. Inoltre, si è resa necessaria una chiara suddivisione tra controller, middleware e routing: una scelta non solo coerente con le best practice di Laravel, ma che consente anche di ottenere un'architettura più solida e a basso accoppiamento. Un esempio concreto è la centralizzazione della logica di verifica dell'autenticazione all'interno di un middleware, evitando duplicazioni di codice; tale aspetto verrà approfondito nel dettaglio successivamente.

Il risultato finale:



Sostituzioni e Miglioramenti delle Servlet

Sulla base dell'analisi precedente, è emerso come nell'implementazione Java ogni funzionalità fosse gestita da una Servlet distinta. Questo approccio, se mantenuto invariato nella migrazione a Laravel, avrebbe portato a una proliferazione di classi poco efficiente e non pienamente allineata con il paradigma del framework. Laravel, infatti, consente di raggruppare all'interno di un'unica classe più azioni logicamente

correlate, permettendo di organizzare in modo più coerente le funzionalità che insistono sullo stesso dominio, pur risolvendo problemi differenti. Questa scelta risulta vantaggiosa non solo in termini di riduzione del numero di classi, ma anche dal punto di vista della leggibilità, della manutenibilità e della corretta mappatura dei concetti di dominio.

In quest'ottica, le Servlet Java dedicate a "login", "logout", "loginFirst" e "registrazione" sono state tradotte in Laravel all'interno di un'unica classe, "Autenticazione", che raccoglie tali funzionalità come metodi distinti. Dal punto di vista logico, inizialmente il comportamento volevamo rimanerlo invariato rispetto all'implementazione originale in Java, ma poi ci siamo accorti che poteva essere migliorato, ma ne parleremo successivamente. A livello strutturale, questa scelta introduce un miglioramento significativo nell'organizzazione del codice. Lo stesso approccio è stato applicato alle Servlet "AccessoAdmin" e "LogoutAdmin", che sono state accorpate nella classe Laravel "AutenticazioneAdmin", contenente i relativi metodi per la gestione dell'autenticazione lato amministratore. Infine, anche le Servlet "*Profilo*" e "*MostraProfilo*" sono state tradotte come metodi all'interno di un'unica classe Laravel, "Profilo", consentendo di mantenere tutte le operazioni legate alla gestione del profilo utente all'interno dello stesso contesto di dominio.

Come già accennato, durante la traduzione la logica dei controller è stata inizialmente mantenuta invariata rispetto all'implementazione Java. Tuttavia, nel corso della migrazione è emersa la possibilità di migliorare ulteriormente la struttura del codice. In particolare, diverse Servlet contenevano porzioni di codice duplicate, come i controlli sull'autenticazione dell'utente.

Per questo motivo, si è scelto di non effettuare una traduzione 1:1 delle Servlet, ma di sfruttare le funzionalità offerte da Laravel introducendo un layer di "**middleware**". I middleware in Laravel implementano un pattern che consente di intercettare la richiesta prima che raggiunga il controller, permettendo di centralizzare logiche trasversali, come autenticazione, autorizzazione e redirezioni.

Sono stati quindi definiti quattro middleware all'interno del package PresentazioneProfilo: **IsAuthenticated**”, **IsAuthenticatedAdmin**”, **RedirectIfAuthenticated**”, **RedirectAdminIfAuthenticated**”

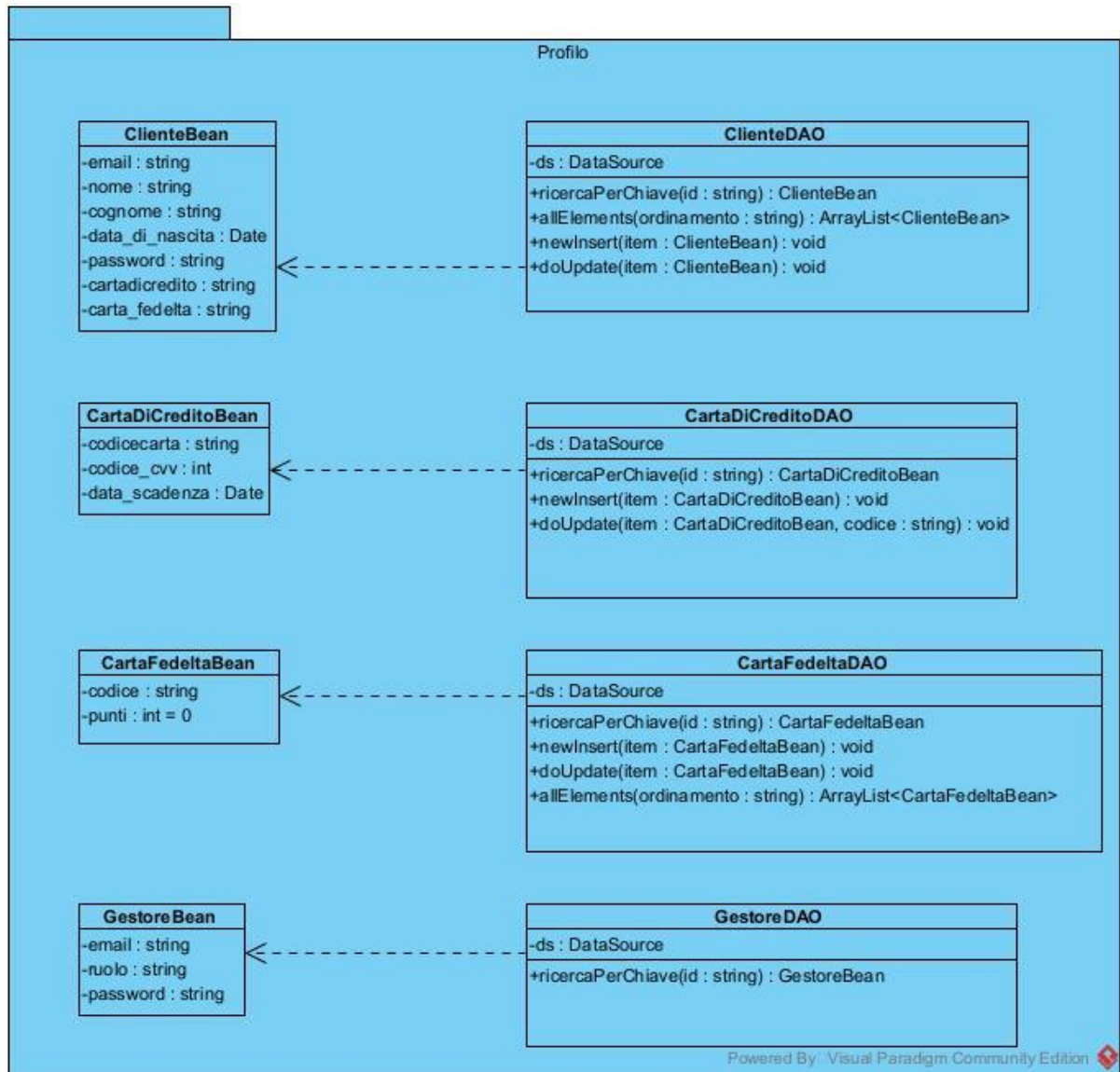
Questi middleware incapsulano le porzioni di codice precedentemente replicate nelle Servlet, migliorando la modularità, riducendo la duplicazione e aumentando la manutenibilità del sistema. Inoltre, questa scelta contribuisce a ottenere un'architettura più pulita e a basso accoppiamento, in linea con le best practice del framework Laravel.

Nella nuova architettura Laravel, i controller si limitano esclusivamente a gestire il flusso della richiesta e della risposta, senza contenere alcun tipo di logica di business o gestione della memoria. Tutta la logica applicativa è stata invece centralizzata all'interno del package **“Profilo”**, che si occupa in modo esplicito delle responsabilità di dominio. Questo aspetto verrà approfondito nelle sezioni successive, dove verrà analizzata nel dettaglio l'organizzazione e il ruolo di tale package.

Sostituzioni e riutilizzo di JSP

Nel package PresentazioneProfilo, facendo riferimento alla documentazione preesistente, sono state individuate e migrate le principali pagine JSP: “login.jsp”, “admin.jsp”, “profilo.jsp” e “registrazione.jsp”. Tali viste sono state tradotte in Laravel come template Blade, rispettivamente “login.blade.php”, “admin.blade.php”, “registrazione.blade.php” e “profilo.blade.php”, introducendo al contempo alcuni miglioramenti grafici e strutturali. La nuova organizzazione delle viste risulta coerente con quanto descritto nella Fase 2, nella quale è stato definito preliminarmente lo scheletro dell'interfaccia tramite il layout principale “app.blade.php”. Questo layout funge da contenitore comune e include le diverse viste interne visualizzate dopo l'autenticazione, come ad esempio “profilo.blade.php”, garantendo una struttura più modulare, riutilizzabile e conforme alle best practice di Laravel.

5.3.2. Package Profilo



In questa fase si è resa necessaria la migrazione del package “**Profilo**”, che, come mostrato in figura, era originariamente composto dalle classi Bean e DAO. Proprio in questo punto si osserva uno dei cambiamenti più significativi rispetto all’implementazione Java, poiché non è stato mantenuto rigidamente il pattern DAO/DTO. Al suo posto è stata adottata una suddivisione in layer separati che distinguono in modo più chiaro la **logica dei model** dalla **logica di business**.

I DAO sono stati trasformati in “**Service**”, mentre i Bean sono stati convertiti in “**modelli Eloquent**”. Questa migrazione non costituisce una traduzione 1:1, in quanto i Bean Java erano semplici Plain Old Java Object, dotati esclusivamente di getter e setter, privi di qualsiasi connessione diretta con il database. Essi rappresentavano unicamente la struttura dei dati, mentre l’accesso al database e l’esecuzione delle query erano interamente demandati ai DAO. Di conseguenza, non era presente una reale logica di business, e alcune porzioni di essa risultavano addirittura collocate nei controller. I modelli Eloquent, rispetto ai Bean, rappresentano un’evoluzione significativa: non sono semplici oggetti passivi, ma classi configurate per interagire direttamente con il database, definendo ad esempio chiavi primarie, tabelle e relazioni. Inoltre, Eloquent consente di modellare relazioni tra entità, aspetto completamente assente nell’implementazione precedente, pur operando in un contesto apparentemente orientato agli oggetti. Va comunque precisato che tali relazioni sono di tipo ORM e non puramente object-oriented. Per ogni entità dati è stato quindi creato il corrispondente “**Service**”, nel quale sono state migrate le funzionalità precedentemente presenti nei DAO. Sebbene i nomi dei metodi rimangano spesso simili, il comportamento risulta differente: i service non si limitano più a eseguire query, ma gestiscono la logica di business, decidendo ad esempio se recuperare un dato dal database o dalla sessione, nonché orchestrando operazioni di inserimento e aggiornamento.

Non tutte le funzioni presenti nei DAO sono state convertite in pura logica di business per due motivi principali:

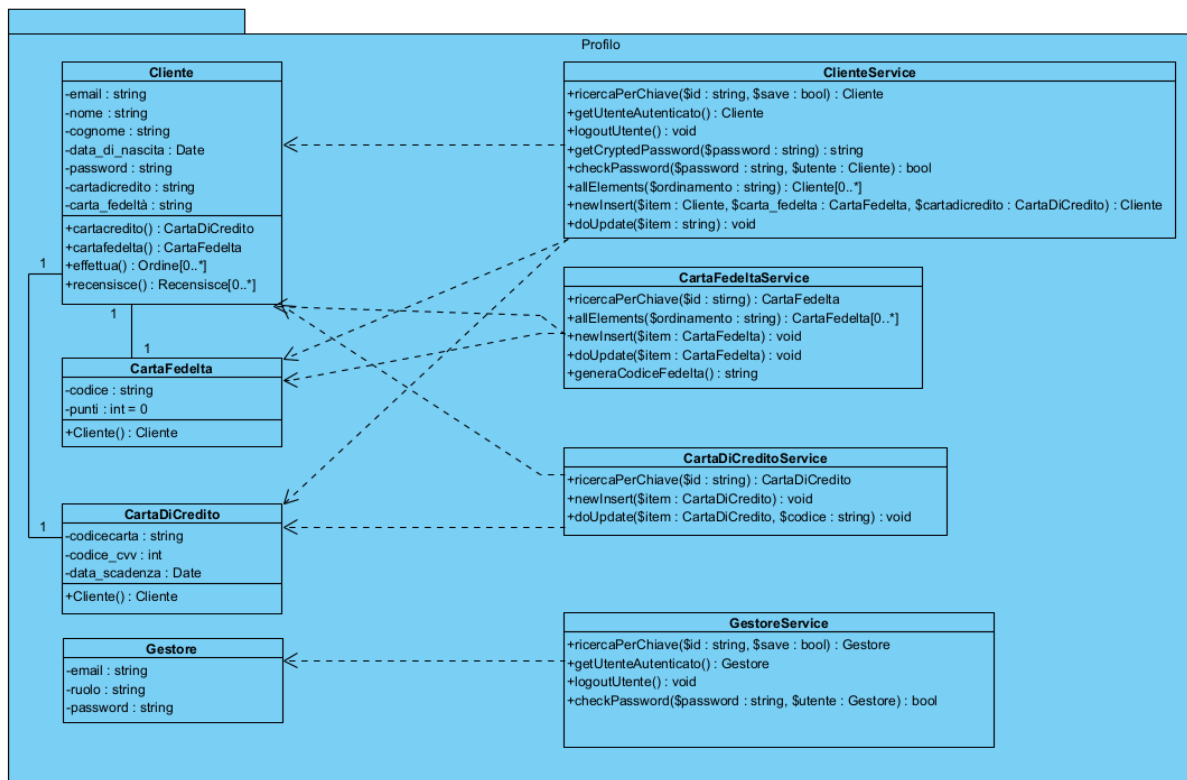
- alcune funzioni non risultavano mai utilizzate nel codice originale e sono state quindi semplicemente tradotte utilizzando i modelli Eloquent, senza introdurre query esplicite;
- il progetto legacy è fortemente “data-driven” e l’obiettivo principale rimane la migrazione: per questo motivo si è evitato di forzare artificialmente la business logic in funzioni che, per loro natura, non la richiedevano.

Inoltre, la migrazione non si è limitata a una semplice riproduzione delle funzionalità esistenti. Dall’analisi delle Servlet è emersa la necessità di

spostare nei service alcune responsabilità precedentemente gestite nei controller, come ad esempio il recupero dell'utente autenticato (indipendentemente dal fatto che provenga dalla sessione o dal database) o l'implementazione di algoritmi di crittografia. Centralizzare tali aspetti nei service consente di migliorare la manutenibilità del sistema: in caso di modifica di un algoritmo di crittografia, ad esempio, l'intervento avverrebbe esclusivamente nel layer di business. Dal punto di vista strutturale, l'architettura risultante non può essere assolutamente definita come service-oriented. Essa rimane concettualmente simile a quella precedente, ma è stata riorganizzata e migliorata per ottenere una separazione più netta delle responsabilità. L'obiettivo principale resta infatti la traduzione del sistema; i miglioramenti introdotti rappresentano un valore aggiunto laddove possibile.

In conclusione, la nuova struttura risulta più ordinata e coerente rispetto alla precedente, pur non stravolgendone radicalmente i principi. I "Service" possono essere visti come un'evoluzione dei vecchi DAO: non eseguono più direttamente query, ma orchestrano l'accesso ai dati tramite i modelli Eloquent e gestiscono in modo più consapevole la business logic, sollevando i controller da responsabilità che non competono al layer di presentazione. Le differenze emergono in particolare nelle interfacce dei service e, più in generale, nell'organizzazione finale del package.

Ecco il risultato finale:



Nuove Class Interfaces

Nome Classe	ClienteService
Descrizione	Classe di business logic che gestisce le operazioni sul profilo Cliente, inclusa l'autenticazione, la cifratura password e orchestrare le varie associazioni con carte fedeltà/credito, decidendo cosa e quando mettere in memoria o meno.
Pre-Condizione	context ClienteService::ricercaPerChiave(string \$id, bool \$save) pre: \$id != null AND \$id != "". context ClienteService::getUtenteAutenticato() pre: Esiste una sessione attiva. context ClienteService::logoutUtente() pre: Esiste una sessione attiva. context ClienteService::getCryptedPassword(string \$password) pre: \$password != null AND \$password != "". context ClienteService::checkPassword(string \$password, Cliente \$utente) pre: \$utente != null AND \$password != "". context ClienteService::allElements(string \$ordinamento) pre: \$ordinamento != null AND \$ordinamento != "" AND \$ordinamento deve rispettare il formato di ORDER BY di SQL ovvero: "parametro asc/desc" dove "parametro" deve corrispondere a un attributo della tabella su cui facciamo l'ordinamento. context ClienteService::newInsert(Cliente \$item, CartaFedelta \$carta_fedelta, CartaDiCredito \$cartadicredito) pre: \$item != null AND \$carta_fedelta != null AND \$cartadicredito != null. context ClienteService::doUpdate(Cliente \$item) pre: \$item != null.
Post-Condizione	context ClienteService::ricercaPerChiave(string \$id, bool \$save) post: Return null OR (\$result == Cliente AND \$result.email == \$id

	AND Return \$result) AND (\$save != false ⇒inserisce \$result nella sessione con chiave “Cliente”) context ClienteService::getUtenteAutenticato() post: Return Cliente dalla sessione corrente OR null se non presente. context ClienteService::logoutUtente() post: L'istanza del Cliente viene rimossa dalla sessione. context ClienteService::getCryptedPassword(string \$password) post: Return \$stringa AND \$stringa.size() = 32 $\wedge$ \$stringa.matches('[0-9a-fA-F]{32}') . context ClienteService::checkPassword(string \$password, Cliente \$utente) post: Return true (<i>/se hash(\$password) == \$utente.password/</i>) OR false. context ClienteService::allElements(string \$ordinamento) post: Return Sequence(Cliente). context ClienteService::newInsert(Cliente \$item, CartaFedelta \$carta_fedelta, CartaDiCredito \$cartadicredito) post: Viene inserita una tupla nel database nella tabella Cliente con I valori che si trovano in \$item AND (\$item.cartafedelta = \$carta_fedelta AND \$item.cartacredito = \$cartadicredito). context ClienteService::doUpdate(Cliente \$item) post: Aggiorna la tupla nel database nella tabella Cliente corrispondente AND Inserisce l'oggetto Cliente memorizzato nella sessione corrente.
Invarianti	

Nome Classe	CartaFedeltaService
Descrizione	Classe di business logic che gestisce il ciclo di vita delle carte fedeltà, la generazione di codici univoci e l'incremento punti.
Pre-Condizione	context CartaFedeltaService::ricercaPerChiave(string \$id) pre: \$id != null AND \$id != "". context CartaFedeltaService::allElements(string \$ordinamento) pre: \$ordinamento != null AND \$ordinamento != "" AND ordinamento deve rispettare il formato di ORDER BY di SQL ovvero: "parametro asc/desc" dove "parametro" deve corrispondere a un attributo della tabella su cui facciamo l'ordinamento. context CartaFedeltaService::newInsert(CartaFedelta \$item) pre: \$item != null. context CartaFedeltaService::doUpdate(CartaFedelta \$item) pre: \$item != null AND \$item->codice != null. context CartaFedeltaService::generaCodiceFedelta() pre: Nessuna pre-condizione
Post-Condizione	context CartaFedeltaService::ricercaPerChiave(string \$id) post: Return CartaFedelta OR null se non esiste. context CartaFedeltaService::allElements(string \$ordinamento) post: Return sequence(CartaFedelta) context CartaFedeltaService::newInsert(CartaFedelta \$item) post: Viene inserita una tupla nel database nella tabella CartaFedelta con I valori che si trovano in \$item. context CartaFedeltaService::doUpdate(CartaFedelta \$item) post: La CartaFedelta corrispondente all'id di \$item ha i punti incrementati di 1. Se il Cliente proprietario è loggato in sessione, la sessione viene aggiornata con la CartaFedelta aggiornata.

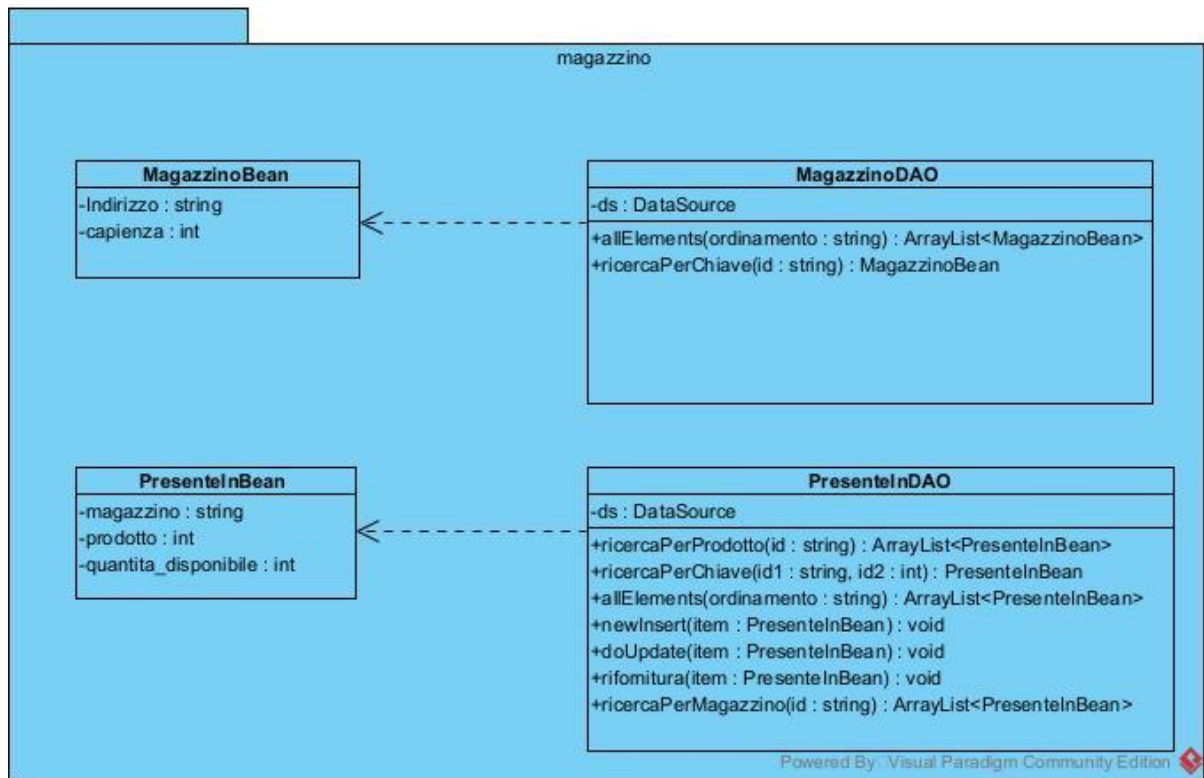
	context CartaFedeltaService::generaCodiceFedelta() post: \$result != null AND \$result.matches('[0-9]+') AND \$result non esiste in alcuna tupla della tabella CartaFedelta AND Return \$result
Invarianti	

Nome Classe	CartaDiCreditoService
Descrizione	Classe di business logic che gestisce il ciclo di vita delle carte di credito e la decisione di quando mettere in memoria le relazioni.
Pre-Condizione	context CartaDiCreditoService::ricercaPerChiave(string \$id) pre: \$id != null AND \$id != "". context CartaDiCreditoService::newInsert(CartaDiCredito \$item) pre: \$item != null. context CartaDiCreditoService::doUpdate(CartaDiCredito \$item, string \$codice) pre: \$item != null AND \$codice != null.
Post-Condizione	context CartaDiCreditoService::ricercaPerChiave(string \$id) post: Return CartaDiCredito OR null se non esiste. context CartaDiCreditoService::newInsert(CartaDiCredito \$item) post: Viene inserita una tupla nel database nella tabella CartaDiCredito con I valori che si trovano in \$item context CartaDiCreditoService::doUpdate(CartaDiCredito \$item, string \$codice) post: La CartaDiCredito corrispondente a \$codice viene aggiornata sul DB con i dati di \$item. Se il Cliente proprietario è loggato in sessione, la sessione viene aggiornata con la CartaDiCredito aggiornata.
Invarianti	

Nome Classe	GestoreService
Descrizione	Classe di business logic che gestisce le operazioni sul Gestore, inclusa l'autenticazione, decidendo cosa e quando mettere in memoria o meno.
Pre-Condizione	context GestoreService::ricercaPerChiave(string \$id, bool \$save) pre: \$id != null AND \$id != "". context GestoreService::getUtenteAutenticato() pre: Esiste una sessione attiva. context GestoreService::logoutUtente() pre: Esiste una sessione attiva. context GestoreService::checkPassword(string \$password, Gestore \$utente) pre: \$password != null AND \$utente != null.
Post-Condizione	context GestoreService::ricercaPerChiave(string \$id, bool \$save) post: Return null OR (\$result == Gestore AND \$result.email == \$id AND Return \$result) AND (\$save != false ⇒ inserisce \$result nella sessione con chiave “Gestore”) context GestoreService::getUtenteAutenticato() post: Return Gestore dalla sessione corrente OR null se non presente. context GestoreService::logoutUtente() post: L'istanza del Gestore viene rimossa dalla sessione. context GestoreService::checkPassword(string \$password, Gestore \$utente) post: Return true (/se <i>hash(\$password) == \$utente.password</i>/) OR false.
Invarianti	

5.4. Fase 4

5.4.1. Package Magazzino



In questa fase si è resa necessaria la migrazione del package **Magazzino**, che, come mostrato in figura, era originariamente composto esclusivamente da classi Bean e DAO. Come già descritto nella fase precedente (fase 3), la migrazione è stata effettuata trasformando i Bean in Model e riconvertendo i DAO in service, orientato il più possibile alla logica di business.

È importante sottolineare che non tutte le funzionalità migrate rappresentano pura logica di business: in alcuni casi sono state effettuate migrazioni 1 a 1, soprattutto per operazioni molto semplici, al fine di non forzare artificialmente l'introduzione di logica di business dove non necessaria. Ogni migrazione di funzione, come in tutte le fasi del progetto, è stata preceduta dall'analisi del codice legacy che ne faceva uso, così da valutare se la funzione potesse essere arricchita con logica di business oppure mantenuta invariata. Le funzioni, infatti, non vengono mai eliminate arbitrariamente: l'obiettivo principale rimane la migrazione, non la rimozione di funzionalità, pertanto tutto il comportamento pre-esistente deve essere preservato e riallocato correttamente nei nuovi layer.

Alcune funzioni, come **ricercaPerMagazzino** e **RicercaPerProdotto**, sono state apparentemente rimosse. In realtà, tali funzionalità risultano ora implicitamente disponibili grazie alle relazioni tra i model introdotte tramite ORM. Nel sistema legacy i Bean non erano realmente object-oriented né collegati tra loro; con la nuova architettura, invece, le associazioni tra model permettono di navigare direttamente tra Magazzino, Prodotto e PresenteIn senza necessità di metodi dedicati di ricerca.

Il service **PresenteIn** assume quindi un ruolo centrale come servizio di coordinamento tra Magazzino e Prodotto, occupandosi di gestire algoritmi e decisioni di business relative alla loro cooperazione e persistenza. Questo è evidente nei nuovi metodi introdotti, tra cui:

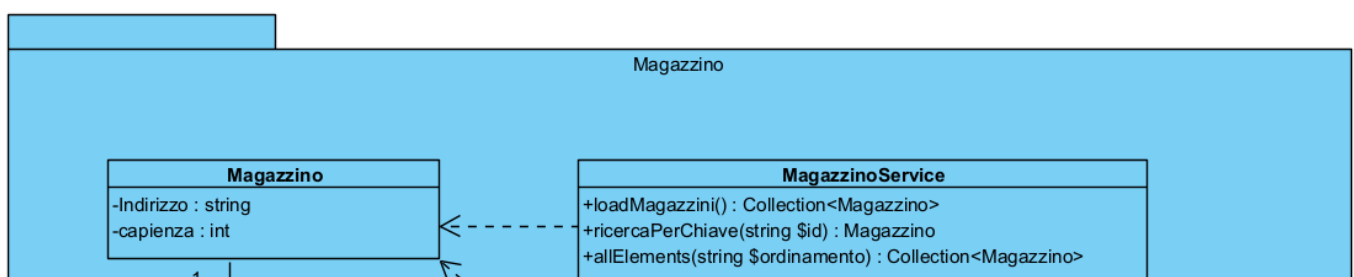
- **getDisponibilità**: consente di calcolare, dato un prodotto, in quali magazzini esso risulta disponibile.
- **getMagazziniDaRifornire**: dato un prodotto e una quantità, determina quali magazzini possono essere riforniti e in che misura, rispettando i vincoli di capienza e applicando regole di distribuzione coerenti (ad esempio: privilegiare magazzini in cui il prodotto è già presente, oppure distribuire la rimanenza su più magazzini se necessario).

Le funzioni **newInsert** e **rifornitura** sono state migrate in modalità 1 a 1. L'analisi del codice ha evidenziato un utilizzo molto limitato di tali metodi e una loro eventuale modifica avrebbe comportato un rischio di forzatura architetturale non giustificato. Pur non essendo idealmente collocate in un service layer, sono state mantenute per non alterare eccessivamente i layer e garantire la piena compatibilità con il legacy. Anche il metodo **allElements** di PresenteIn è stato migrato 1 a 1. Dall'analisi delle servlet legacy è emerso che, nel nuovo sistema, questo metodo non verrà mai utilizzato, tuttavia, è stato mantenuto per completare l'obiettivo di migrazione e preservare tutte le funzionalità storiche. Eventuali miglioramenti futuri potranno essere affrontati tramite successive change request orientate esclusivamente al perfezionamento del layer di business. Sono inoltre, stati introdotti altri nuovi metodi:

- **quantitàTotaleProdotto:** calcola la quantità totale di un prodotto considerando tutti i magazzini.
- **doUpdate:** anche se potrebbe sembrare già presente, in realtà gestisce il decremento della quantità acquistata di un prodotto, distribuendolo potenzialmente su più magazzini. Pur contenendo una logica di business minima, verifica almeno che l'aggiornamento non porti la quantità sotto zero prima di applicarlo.

Conclusa la migrazione di `PresenteIn`, si è proceduto alla migrazione di `MagazzinoDAO` in **MagazzinoServices**. Questa classe non rappresenta più un semplice DAO, poiché non si limita a effettuare chiamate al database, ma gestisce anche una cache dei magazzini. In base a regole di business definite, è stato stabilito un numero massimo plausibile di magazzini e una bassa frequenza di aggiornamento, con possibilità di modifica riservata esclusivamente al Database Administrator. Si è quindi deciso di mantenere tutti i magazzini in cache, senza memorizzare anche le relazioni. Tale scelta è giustificata dal fatto che l'acquisizione di nuovi magazzini è un evento raro e al di fuori dello scope operativo quotidiano della piattaforma. Anche in scenari con alcune centinaia di magazzini, la soluzione risulta pienamente sostenibile. Dal punto di vista delle firme dei metodi, **MagazzinoServices** risulta molto simile al precedente **MagazzinoDAO**, ma il comportamento interno è cambiato. La migrazione può quindi considerarsi non solo riuscita, ma anche migliorativa, grazie all'introduzione di logica di business, con anche meccanismi di caching.

Ecco il risultato finale:



Nuove Class Interfaces

Nome Classe	MagazzinoService
Descrizione	Classe di business logic che gestisce la lettura e la cache dei magazzini, fornendo metodi per la ricerca rapida dei magazzini, ottimizzando le operazioni di accesso ai dati tramite caching.
Pre-Condizione	context MagazzinoService::loadMagazzini() pre: Nessuna pre-condizione context MagazzinoService::ricercaPerChiave(string \$id) pre: \$id != null AND \$id != "". context MagazzinoService::allElements(string \$ordinamento) pre: \$ordinamento != null AND \$ordinamento != "" AND ordinamento deve rispettare il formato di ORDER BY di SQL ovvero: “parametro asc/desc” dove “parametro” deve corrispondere a un attributo della tabella su cui facciamo l’ordinamento.
Post-Condizione	context MagazzinoService::loadMagazzini() post: Return sequence(Magazzino). context MagazzinoService::ricercaPerChiave(string \$id) post: Return Magazzino OR Null se non esiste. context MagazzinoService::allElements(string \$ordinamento) post: Return Sequence(Magazzini)
Invarianti	

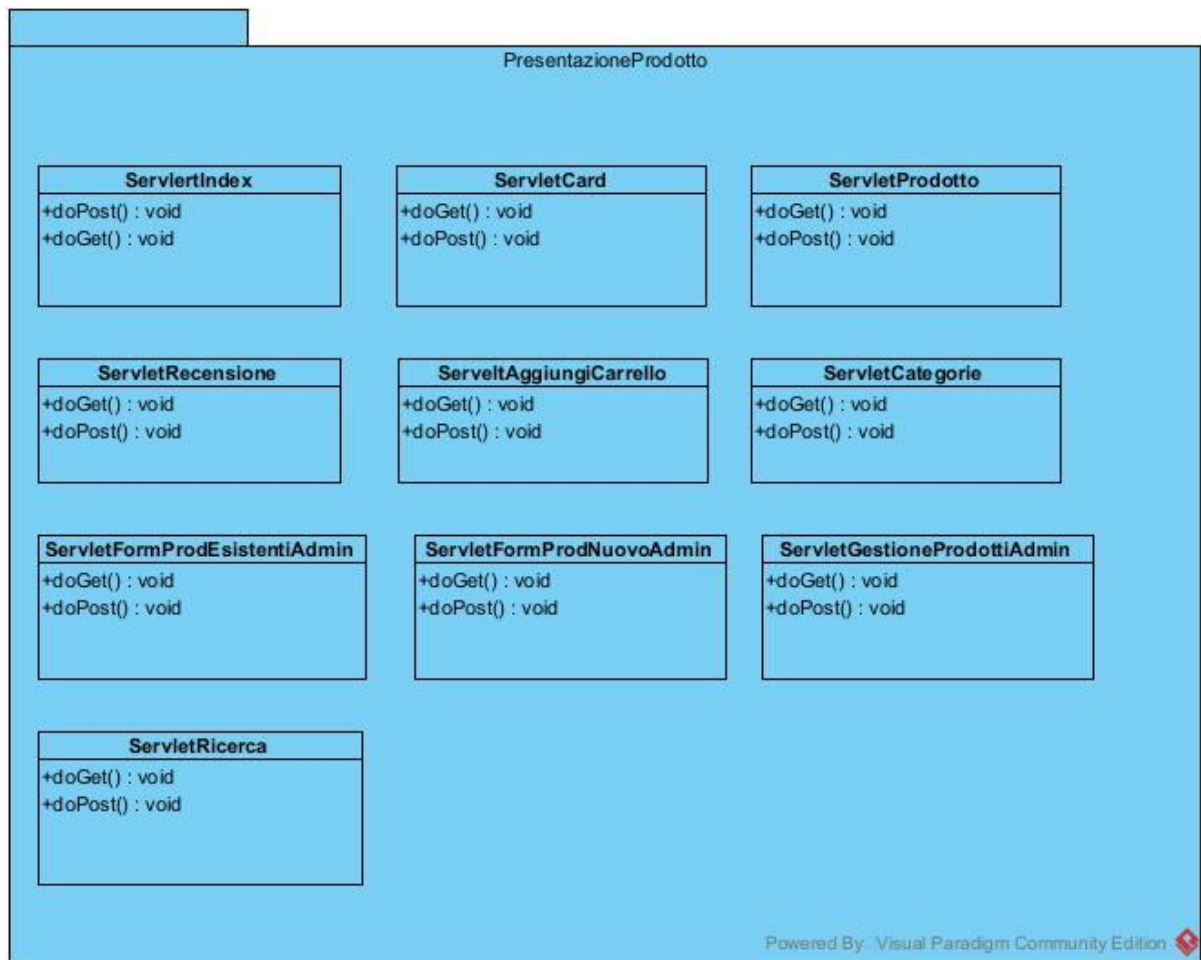
Nome Classe	PresenteInService
Descrizione	Classe di business logic che gestisce la disponibilità dei prodotti nei magazzini, il rifornimento, l'aggiornamento delle giacenze e la distribuzione dei prodotti tra i vari magazzini.
Pre-Condizione	context PresenteInService::ricercaPerChiave(int \$id1, string \$id2) pre: \$id2 != null AND \$id2 != "" AND \$id1 >= 0. context PresenteInService::getMagazziniDaRifornire(Prodotto \$prodotto, int \$quantita) pre: \$prodotto != null AND \$prodotto.codice_prodotto != null AND \$prodotto.codice_prodotto != "" AND \$quantita > 0 AND la quantità da aggiungere non supera mai la capienza dello specifico magazzino. context PresenteInService::getDisponibilita(Prodotto \$prodotto, int \$quantitaDaAcquistare) pre: \$prodotto != null AND \$prodotto.codice_prodotto != null AND \$prodotto.codice_prodotto != "" AND \$quantitaDaAcquistare > 0. context PresenteInService::quantitaTotaleProdotto(int \$codiceProdotto) pre: Nessuna pre-condizione. context PresenteInService::newInsert(PresenteIn \$item) pre: \$item != null. context PresenteInService::rifornitura(PresenteIn \$item) pre: \$item != null. context PresenteInService::doUpdate(PresenteIn \$item, int \$quantita) pre: \$item != null AND \$quantita > 0 AND ((\$item.quantita_disponibile + \$quantita) <= \$item.getMagazzino.capienza). context PresenteInService::allElements(string \$ordinamento) pre: \$ordinamento != null AND \$ordinamento != "" AND ordinamento deve rispettare il formato di ORDER BY di SQL ovvero: "parametro asc/desc" dove "parametro" deve

	corrispondere a un attributo della tabella su cui facciamo l'ordinamento.
Post-Condizione	context PresenteInService::ricercaPerChiave(int \$id1 string \$id2) post: Return null OR (\$result == PresenteIn AND \$result.prodotto == \$id1 AND \$result.magazzino == \$id2) context PresenteInService::getMagazziniDaRifornire(Prodotto \$prodotto int \$quantita) post: Return Sequence(\$result) AND (\$result è un array associativo AND ogni elemento di \$result ha: - 'magazzino' == istanza di Magazzino AND la quantità da aggiungere non è maggiore della capienza per quel magazzino - 'quantita' > 0 - 'presente' == 0 OR 1 AND la somma delle 'quantita' in \$result == \$quantita AND \$result->forAll(e e.quantita <= e.magazzino.capienza)) context PresenteInService::getDisponibilita(Prodotto \$prodotto int \$quantitaDaAcquistare) post:Return Sequence(\$result) AND \$result è un array associativo AND (ogni elemento di \$result ha: - 'presente_in' == istanza di PresenteIn associata con quel \$prodotto

	- 'quantita' >= 0 - 'quantita' <= 'presente_in' -> quantita_disponibile) AND \$result->forAll(e e.quantita <= \$quantitaDaAcquistare)) context PresenteInService::quantitaTotaleProdotto(int \$codiceProdotto) post: Viene restituito la somma delle disponibilita in tutti i magazzini del prodotto che ha come codice \$codiceProdotto OR o context PresenteInService::newInsert(PresenteIn \$item) post: Viene inserita una tupla nel database nella tabella PresenteIn con i valori che si trovano in \$item context PresenteInService::rifornitura(PresenteIn \$item) post: Viene aggiornato il campo quantita_disponibile all'interno di una tupla nel database della tabella PresenteIn con chiavi presenti in \$item context PresenteInService::doUpdate(PresenteIn \$item int \$quantita) post: Il PresenteIn corrispondente a \$item viene decrementato la quantita in base al valore presente in \$quantita. context PresenteInService::allElements(string \$ordinamento) post:Return sequence(PresenteIn)
Invarianti	

5.5. Fase 5

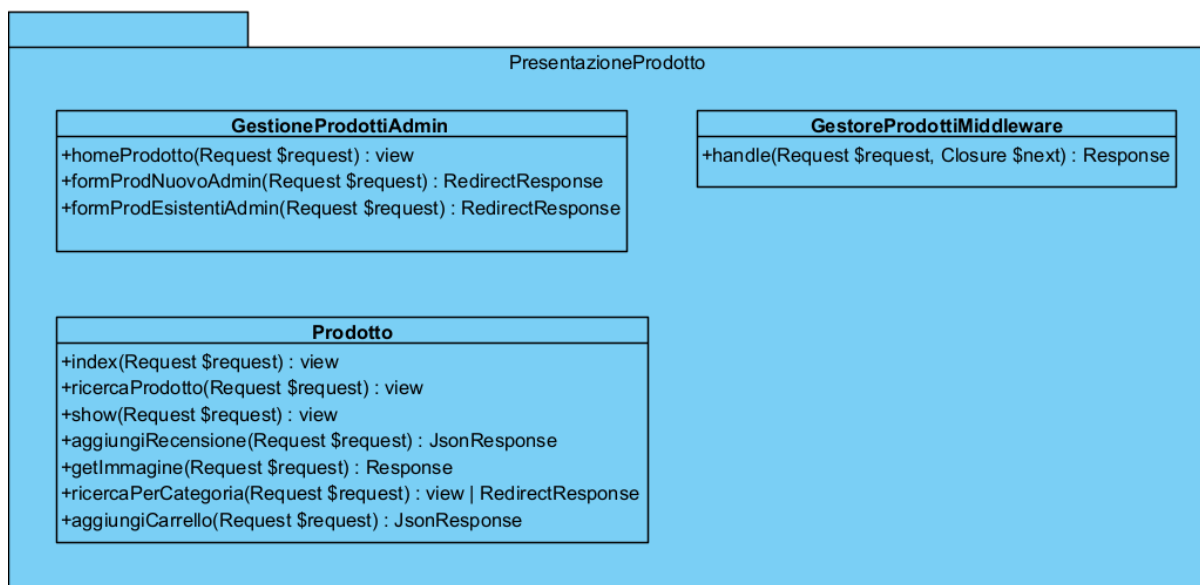
5.5.1. Package PresentazioneProdotto



In questa fase della migrazione, come preventivato nei punti precedenti, si doveva intervenire sia sul package **Prodotto** e sia sul package **PresentazioneProdotto**. Discutendo prima il package **PresentazioneProdotto**, è stata condotta un'analisi approfondita dei componenti presenti nel sistema legacy, costituiti dalle relative pagine JSP e Servlet (come riportato in figura), esso infatti viene identificato all'interno del layer di presentazione. L'analisi ha consentito di individuare con chiarezza le responsabilità dei singoli componenti e i diversi flussi applicativi gestiti dal sistema. Tra questi rientrano: la visualizzazione dei sei top prodotti nella home page; le differenti modalità di ricerca e filtraggio dei prodotti per categoria e sottocategoria; la consultazione dei prodotti in sconto; l'accesso alla pagina di dettaglio del singolo prodotto, comprensiva di tutte le informazioni descrittive, con possibilità di inserire recensioni e aggiungere il prodotto al carrello; infine, le funzionalità dedicate all'area riservata all'amministratore per l'inserimento e la gestione di prodotti nuovi o già esistenti.

Questa fase non si è limitata a una semplice traduzione tecnologica verso il nuovo framework, ma ha rappresentato un vero e proprio processo di riprogettazione all'interno di Laravel, sfruttando un'architettura maggiormente coerente con i principi di separazione delle responsabilità e della business logic. Dall'analisi del sistema legacy è infatti emerso che i controller accentravano gran parte della logica applicativa, senza un'adeguata suddivisione dei compiti tra i diversi livelli dell'architettura. Ciò ha fatto presumere l'assenza, o quantomeno una presenza marginale, del package **Prodotto**. Proprio per questo motivo, dopo aver ridefinito in modo strutturato il layer di presentazione, si è potuto intervenire anche sul package "Prodotto", riorganizzandolo secondo una separazione più netta tra presentazione, controllo e dominio, in linea con le buone pratiche architetturali adottate nel nuovo contesto applicativo.

Il risultato finale:



Sostituzioni e Miglioramenti delle Servlet

Si parte col dire che dall'analisi è emerso che alcune di esse non erano strettamente legate alla gestione del prodotto in sé, bensì a funzionalità connesse al processo di acquisto e alla gestione degli ordini successiva all'acquisto.

In particolare, le classi:

- ServletGestioneOrdiniAdmin.java
- ServletFormOrdiniAdmin.java

pur essendo inserite nel package relativo al prodotto, risultavano di fatto orientate alla gestione degli ordini. Per questo motivo si è deciso di ricollocarle nel modulo **Acquisto**, ritenuto architetturealmente più coerente rispetto alle responsabilità effettivamente svolte. Questa scelta ha permesso di ottenere una maggiore coerenza strutturale tra i moduli applicativi, separando in modo più netto la logica del dominio Prodotto da quella relativa al processo di acquisto e alla gestione degli ordini. Inoltre, tale suddivisione migliora la manutenibilità e la testabilità del sistema, poiché il modulo Acquisto potrà essere sviluppato ed evoluto in modo autonomo.

Non solo le classi Java, ma anche le relative JSP e le pagine collegate alla gestione degli ordini devono essere ricollocate dal package **Prodotto** al package **Acquisto**, in coerenza con la nuova suddivisione delle responsabilità, infatti, le funzionalità di gestione ordini erano in realtà implementate nella **paginaGestioneOrdini.jsp**, separata dalla pagina dedicata alla rifornimento dei prodotti. Di conseguenza, oltre allo spostamento delle servlet relative alla gestione ordini, si è reso necessario ricollocare anche la **paginaGestioneOrdini.jsp** dal package Prodotto al package Acquisto.

Continuando con la migrazione, bisogna procedere con:

- ServletGestioneProdottiAdmin.java
- ServletFormProdNuovoAdmin.java
- ServletFormProdEsistentiAdmin.java
- ServletIndex.java
- ServletRicerca.java
- ServletCategorie.java
- ServletProdotto.java
- ServletCard.java
- ServletRecensione.java

- ServletAggiungiAlCarrello.java

Per la loro migrazione in Laravel si è scelto di non effettuare una conversione uno-a-uno in controller distinti, bensì di riorganizzare le responsabilità in modo più coerente dal punto di vista architetturale, creando complessivamente due controller principali.

Il primo controller Laravel prende il nome di **Prodotto** e racchiude al suo interno una serie di funzioni, ciascuna delle quali mappa una singola servlet tra le seguenti:

- ServletIndex.java
- ServletRicerca.java
- ServletCategorie.java
- ServletProdotto.java
- ServletCard.java
- ServletRecensione.java
- ServletAggiungiAlCarrello.java

In questo modo, tutte le funzionalità legate alla visualizzazione, ricerca, dettaglio del prodotto, gestione recensioni e aggiunta al carrello sono state accorpate in un unico controller coerente con il dominio di Prodotto. Il secondo controller Laravel è stato denominato **GestioneProdottiAdmin** e raccoglie invece le funzionalità relative alla gestione e alla rifornimento dei prodotti. Al suo interno sono state definite tre funzioni, ciascuna mappata rispettivamente a:

- ServletGestioneProdottiAdmin.java
- ServletFormProdNuovoAdmin.java
- ServletFormProdEsistentiAdmin.java

Dal diagramma riportato in precedenza è possibile osservare nel dettaglio la corrispondenza tra le servlet legacy e le nuove funzioni dei controller Laravel; già dalla denominazione dei metodi è intuibile quale servlet sia stata mappata e con quale responsabilità. Una volta completata la mappatura delle classi, i controller sono stati riorganizzati eliminando completamente la logica di business che nel sistema legacy

era contenuta al loro interno. Tale logica è stata spostata all'interno di appositi **Service**. Di conseguenza, i nuovi controller risultano più puliti, leggeri e aderenti al principio di separazione delle responsabilità, occupandosi esclusivamente della gestione delle richieste HTTP e del coordinamento con il livello di servizio. Un ulteriore intervento significativo all'interno del modulo **PresentazioneProdotto** ha riguardato la gestione delle autorizzazioni. Nel sistema legacy, le servlet dedicate alla gestione e rifornimento dei prodotti contenevano ripetutamente codice interno per verificare che l'utente fosse autorizzato a eseguire determinate operazioni (ad esempio controllando che fosse amministratore di prodotti e non di ordini). Nel nuovo sistema tale logica è stata rimossa dai controller e centralizzata attraverso la creazione di un **middleware** specifico, chiamato **GestoreProdottiMiddleware**. Questo middleware non verifica non che l'utente sia autenticato e amministratore (poiché è un controllo già introdotto nelle fasi precedenti), ma esso controlla che l'utente possieda effettivamente il ruolo abilitato alla gestione dei prodotti. Il middleware viene applicato esclusivamente alle rotte interessate, garantendo un controllo centralizzato, riutilizzabile e coerente, eliminando duplicazioni di codice e migliorando ulteriormente la pulizia architetturale del sistema.

Sostituzioni e riutilizzo di JSP

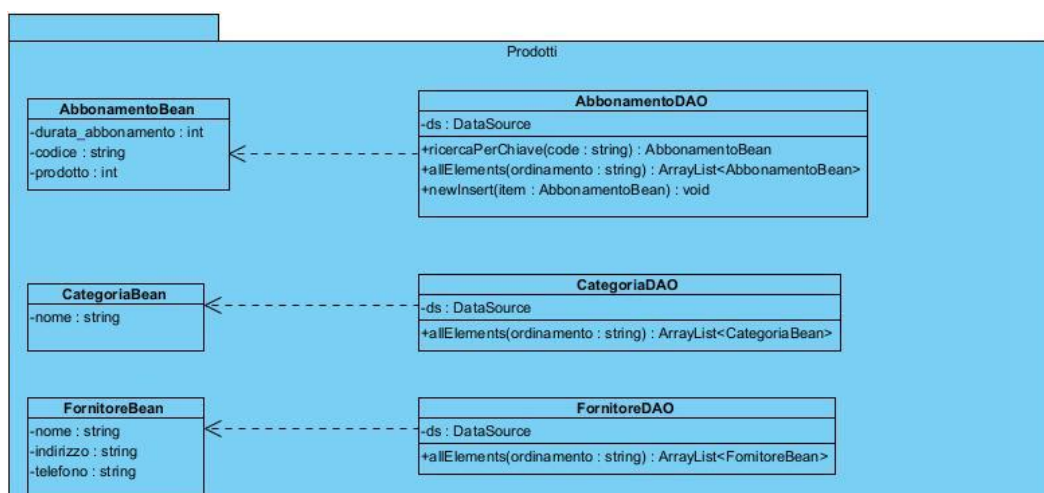
Per quanto riguarda le JSP, come già anticipato nel punto precedente, la **paginaGestioneOrdini.jsp** è stata ricollocata nell'altro modulo e verrà sviluppata nella fase successiva, dedicata al modulo "Acquisto". In questa fase, invece, si è proceduto alla migrazione delle seguenti pagine:

- **paginaGioco.jsp**
- **paginaRicerca.jsp**
- **homepage.jsp**
- **paginaAmministratore.jsp**

La migrazione ha mantenuto la stessa denominazione dei file, modificandone unicamente l'estensione in **.blade.php**. In questo caso si è accorti che era possibile effettuare una migrazione sostanzialmente uno-a-uno, trasformando il codice JSP in sintassi Blade/PHP e

adattando le direttive proprie dell'ambiente Java al nuovo framework. Pur cercando di preservare il più possibile la struttura originale, inclusi CSS e JavaScript, la migrazione non si è limitata a una conversione meccanica. Sono stati infatti apportati alcuni miglioramenti puntuali, sia a livello di organizzazione del markup sia in termini di responsività e ottimizzazione dell'interfaccia, al fine di rendere le pagine maggiormente coerenti con le buone pratiche attuali e con l'architettura complessiva del nuovo sistema. In questo modo, anche il layer di presentazione è stato riallineato non solo tecnologicamente, ma anche qualitativamente rispetto al sistema legacy, anche se di poco.

5.5.2. Package Prodotti



Dopo aver completato la migrazione del package **PresentazioneProdotto**, si è reso necessario intervenire sul package **Prodotto**. Come già evidenziato nei punti precedenti, questo modulo, analogamente ai moduli di Profilo e Magazzino, rappresenta un livello legato alla logica di dominio; tuttavia, nel sistema legacy non racchiudeva effettiva business logic, ma svolgeva prevalentemente il ruolo di DAO/DTO, limitandosi alla gestione dell'accesso ai dati e al

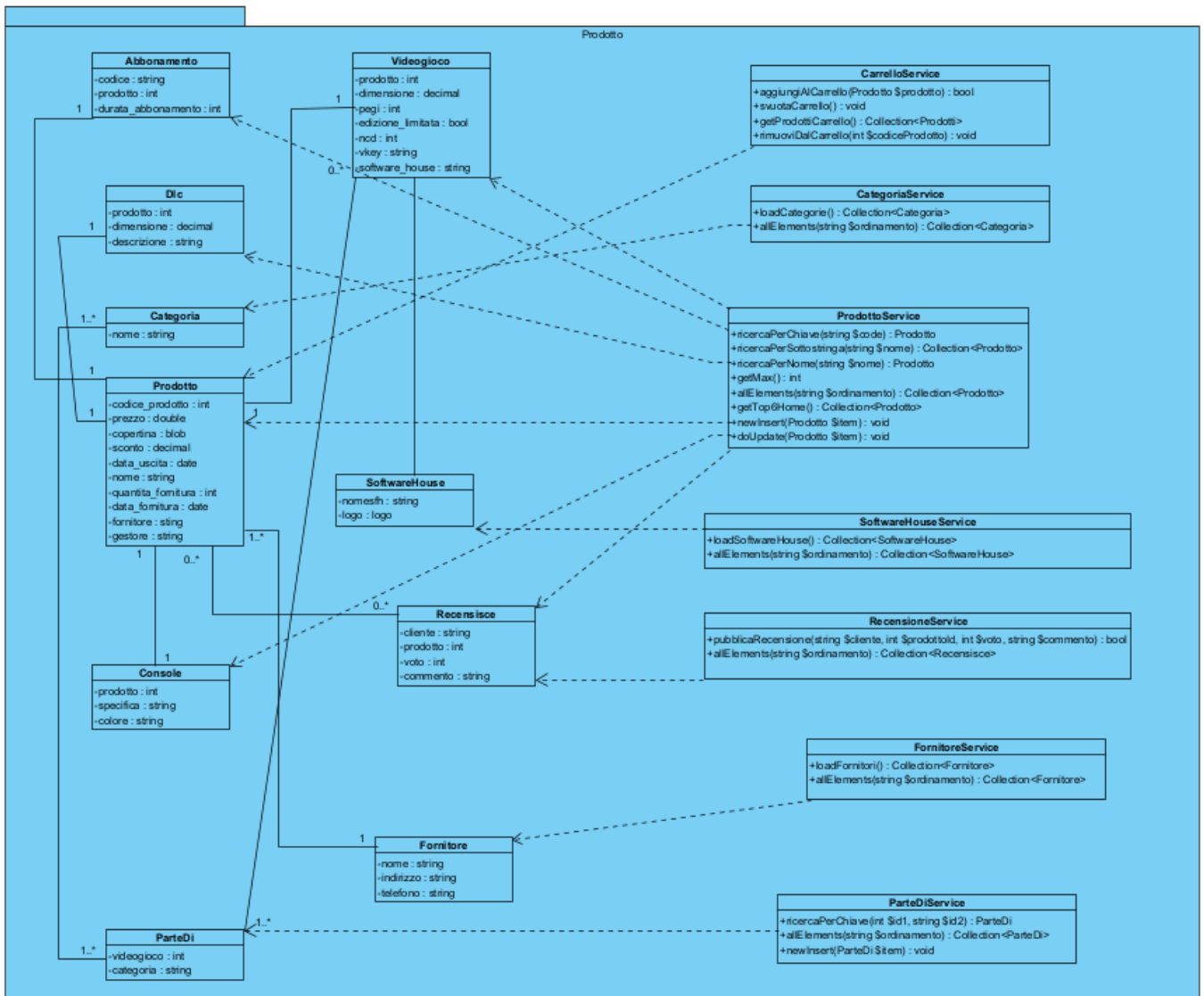
trasferimento delle informazioni. Coerentemente con l'approccio adottato nelle fasi precedenti, tutti i bean sono stati trasformati in Model Laravel, arricchiti con le opportune relazioni ORM. I DAO, invece, sono stati migrati in Service, con l'obiettivo di accorpare al loro interno la business logic e separarla definitivamente dal livello di presentazione e dai controller. Come già accaduto negli altri moduli, la migrazione delle funzioni è stata effettuata seguendo criteri differenti a seconda dei casi. In alcune situazioni la migrazione è avvenuta in modo uno-a-uno: ciò è accaduto quando una determinata funzione era poco utilizzata o aveva una responsabilità ben circoscritta. In questi casi, pur non essendo presente una forte componente di business logic (trattandosi spesso di semplici operazioni sui dati), si è scelto di mantenerle per non perdere funzionalità e per preservare una buona separazione strutturale, evitando comunque di inserire direttamente query puro SQL. In altri casi, alcune funzioni o interi DAO non sono stati migrati esplicitamente, infatti, questa scelta non contraddice l'obiettivo di non perdere funzionalità durante la migrazione, al contrario, si è constatato che, grazie all'introduzione delle relazioni ORM tra i Model, determinate operazioni potevano essere ottenute semplicemente accedendo alle relazioni stesse, senza la necessità di mantenere metodi specifici. In questo modo non si perde alcuna funzionalità, bensì si semplifica e si snellisce il codice, riducendo complessità e ridondanze e migliorando anche la testabilità. Vi sono poi funzioni che hanno mantenuto lo stesso nome, ma il cui comportamento è stato parzialmente modificato per renderle più aderenti a una reale logica di business, spostando al loro interno controlli e operazioni che nel sistema legacy erano distribuiti in modo poco strutturato. Infine, sono state introdotte anche nuove funzioni all'interno dei Service. Questo è stato necessario perché, come descritto nella fase precedente, tutta la business logic precedentemente contenuta nei controller è stata rimossa. Tale logica è stata analizzata, riorganizzata e suddivisa in modo coerente, dando origine a metodi strutturati che rappresentano veri e propri algoritmi. Il risultato complessivo è un package Prodotto che non si limita più a svolgere un ruolo di semplice accesso ai dati, ma che incarna effettivamente la logica di dominio, in linea con una corretta separazione delle responsabilità e con le buone pratiche architetturali adottate nel nuovo sistema, un esempio è proprio la funzione su come vengono gestiti e ottenuti i top 6 prodotti.

Nella presente overview non si entrerà nel dettaglio puntuale di ogni singola funzione del package Prodotto, poiché le modifiche relative ai tipi di ritorno, alle firme dei metodi e ai cambiamenti comportamentali rispetto al sistema legacy sono formalmente documentate nei diagrammi UML, nelle class interface e nelle specifiche OCL. Il confronto diretto tra la versione precedente e quella attuale consente infatti di cogliere in modo rigoroso le differenze strutturali e semantiche introdotte dalla migrazione. È tuttavia utile fornire un quadro generale delle principali scelte architetturali adottate.

Un primo aspetto riguarda alcuni Service che nel sistema legacy presentavano un comportamento sostanzialmente statico, analogamente a quanto riscontrato nel package Magazzino. In questi casi, pur mantenendo invariati i nomi delle funzioni, è stata modificata la modalità di gestione dei dati introducendo un meccanismo di cache. Le informazioni non vengono più recuperate sistematicamente dal database, ma memorizzate temporaneamente, con una gestione della scadenza e della successiva invalidazione. Solo al verificarsi di determinate condizioni, come la scadenza temporale o l'aggiornamento dei dati, si procede a un nuovo accesso al database. Questo approccio è stato applicato, ad esempio, nella gestione delle categorie e delle software house. Un intervento particolarmente rilevante ha riguardato i Service dedicati a videogiochi, DLC, console e abbonamenti. Nel sistema precedente tali entità erano gestite da Service distinti, tuttavia, è stato deciso di eliminarli e accentrare la gestione all'interno di un unico service **Prodotto**. La motivazione risiede nel fatto che tali entità rappresentano specializzazioni del concetto di prodotto e, pertanto, risultava architetturalmente più corretto che fosse esso a coordinarne la gestione. Anche in questo caso alcune funzionalità sono state rimosse grazie all'utilizzo delle relazioni ORM, mentre altre sono state modificate introducendo vera business logic, ad esempio: sono state definite regole più precise su cosa e quando debba essere salvato in memoria, anche tramite tecniche di eager loading, al fine di evitare query ripetute e migliorare le performance complessive. Particolarmente significativa è la funzionalità relativa ai top sei prodotti, utilizzata per la dashboard della homepage. Tale logica non rappresenta una semplice query, ma un vero e proprio calcolo di dominio: i prodotti vengono classificati secondo criteri differenti (ad esempio miglior recensito, maggiormente scontato,

ecc.) e da questi calcoli vengono determinati i sei elementi da mostrare. Questa funzionalità fa uso estensivo della cache, con una durata sufficientemente ampia da evitare accessi frequenti al database. Tuttavia, sono stati introdotti meccanismi di invalidazione della cache in corrispondenza delle operazioni di save e update sul prodotto. Qualora un prodotto venga aggiornato o ne venga inserito uno nuovo (ad esempio in caso di rifornimento), la cache dei “top 6” viene invalidata, poiché la classifica potrebbe variare. È importante sottolineare che, pur mantenendo in alcuni casi lo stesso nome delle funzioni rispetto al sistema precedente (ad esempio nei metodi di salvataggio e aggiornamento), il comportamento interno è stato arricchito con decisioni di business e controlli aggiuntivi. Questo dimostra come la migrazione non sia stata una mera trasposizione tecnica.

Ecco il risultato finale:



Nuove Class Interfaces

Nome Classe	ProdottoService
Descrizione	Classe di business logic che gestisce le operazioni sui Prodotti, che siano Videogiochi, Console, DLC o Abbonamenti.
Pre-Condizione	context ProdottoService::ricercaPerChiave(string \$code) pre: \$code != null AND \$code != "". context ProdottoService::ricercaPerSottostringa(string \$nome) pre: \$nome != null. context ProdottoService::ricercaPerNome(string \$nome) pre: \$nome != null. context ProdottoService::getMax() pre: Nessuna precondizione". context ProdottoService::allElements(string \$ordinamento) pre: \$ordinamento != null AND \$ordinamento != "" AND \$ordinamento deve rispettare il formato di ORDER BY di SQL ovvero: "parametro asc/desc" dove "parametro" deve corrispondere a un attributo della tabella su cui facciamo l'ordinamento. context ProdottoService::getTop6Home() pre: Esiste almeno una recensione nel database AND Esistono almeno 2 Videogiochi nel database AND Esistono almeno 4 Videogiochi con sconto > 0 context ProdottoService::newInsert(Prodotto \$item) pre: \$item != null AND ((\$item.videogioco != null AND \$item.console == null AND \$item.dlc == null AND \$item.abbonamento == null) OR (\$item.videogioco == null AND \$item.console != null AND \$item.dlc == null AND \$item.abbonamento == null) OR (\$item.videogioco == null AND \$item.console == null AND \$item.dlc != null AND \$item.abbonamento == null) OR (\$item.videogioco == null AND \$item.console == null AND \$item.dlc == null AND \$item.abbonamento != null)) context ProdottoService::doUpdate(Prodotto \$item) pre: \$item != null AND Esiste nel database una tupla Prodotto

	con codice_prodotto = \$item.codice_prodotto AND ((\$item.videogioco != null AND \$item.console == null AND \$item.dlc == null AND \$item.abbonamento == null) OR (\$item.videogioco == null AND \$item.console != null AND \$item.dlc == null AND \$item.abbonamento == null) OR (\$item.videogioco == null AND \$item.console == null AND \$item.dlc != null AND \$item.abbonamento == null) OR (\$item.videogioco == null AND \$item.console == null AND \$item.dlc == null AND \$item.abbonamento != null))
Post-Condizione	context ProdottoService::ricercaPerChiave(string \$code) post: Return null OR (\$result == Prodotto AND \$result.codice_prodotto == \$code AND (\$result.prezzo_effettivo = (\$result.sconto > 0 ? \$result.prezzo * (1 - \$result.sconto / 100) : \$result.prezzo)) AND \$result.disponibile = (\$result.quantita_fornitura > 0) AND \$result.necessita_rifornimento = (\$result.quantita_fornitura < 10)). context ProdottoService::ricercaPerSottostringa(string \$nome) post: Return Collection(Prodotto) AND Per ogni Prodotto p in Return vale: esiste nel database una tupla Prodotto con codice_prodotto = p.codice_prodotto AND p.nome contiene la sottostringa \$nome AND p.prezzo_effettivo = (p.sconto > 0 ? p.prezzo * (1 - p.sconto / 100) : p.prezzo) AND p.disponibile = (p.quantita_fornitura > 0) AND p.in_promozione = (p.sconto > 0). context ProdottoService::ricercaPerNome(string \$nome) post: Return null OR (Return \$result AND \$result = Prodotto AND esiste nel database una tupla Prodotto con nome = \$nome AND \$result.nome == \$nome AND \$result.necessita_rifornimento = (\$result.quantita_fornitura < 10) AND (\$result.data_fornitura != null ? \$result.giorni_data_fornitura = differenza in giorni tra data corrente e \$result data_fornitura : \$result.giorni_data_fornitura = null)). context ProdottoService::getMax() post: Return \$result AND (\$result = (massimo valore codice_prodotto nella tabella Prodotto) + 1 OR \$result = 1). context ProdottoService::allElements(string \$ordinamento)

	post: Return collection(Prodotto). context ProdottoService::getTop6Home() post: Return Collection(Prodotto) di esattamente 6 prodotti AND (\$migliorRecensito = prodotto con la media recensioni più alta AND \$ultimoUscito = videogioco più recente escluso \$migliorRecensito AND \$quattroScontati = primi 4 videogiochi con sconto > 0 esclusi \$migliorRecensito e \$ultimoUscito AND Collection(Prodotto) = (\$migliorRecensito + \$ultimoUscito + \$quattroScontati)) AND Per ogni Prodotto p in Collection(Prodotto) vale: p.codice_prodotto esiste nel database AND p.prezzo_effettivo = (p.sconto > 0 ? p.prezzo * (1 - p.sconto / 100) : p.prezzo) AND p.disponibile = (p.quantita_fornitura > 0) AND p.in_promozione = (p.sconto > 0). context ProdottoService::newInsert(Prodotto \$item) post: Viene inserita una tupla nel database nella tabella Prodotto con i valori che si trovano in \$item AND viene inserita la tupla corrispondente alla sua specializzazione (Videogioco, Console, DLC o Abbonamento) AND la cache 'top6_home' viene invalidata. context ProdottoService::doUpdate(Prodotto \$item) post: La tupla nella tabella Prodotto corrispondente a \$item viene aggiornata con i nuovi valori AND viene aggiornata la tupla della sua specializzazione (Videogioco, Console, DLC o Abbonamento) AND la cache 'top6_home' viene invalidata
Invarianti	

Nome Classe	RecensisceService
Descrizione	Classe di business logic che gestisce le operazioni sulle Recensioni.
Pre-Condizione	context RecensisceService::pubblicaRecensione(string \$cliente, int \$prodottoId, int \$voto, string \$commento) pre: \$cliente != null AND \$prodottoId != null AND \$voto != null AND \$commento != null. context RecensisceService::allElements(string \$ordinamento) pre: \$ordinamento != null AND \$ordinamento != "" AND \$ordinamento deve rispettare il formato di ORDER BY di SQL ovvero: "parametro asc/desc" dove "parametro" deve corrispondere a un attributo della tabella su cui facciamo l'ordinamento.
Post-Condizione	context RecensisceService::pubblicaRecensione(string \$cliente, int \$prodottoId, int \$voto, string \$commento) post: (Return true AND viene inserita nel database una tupla Recensisce con Recensisce.cliente = \$cliente AND Recensisce.prodotto = \$prodottoId AND Recensisce.voto = \$voto AND Recensisce.commento = \$commento) OR (Return false se esiste già una recensione nel database con Recensisce.cliente = \$cliente AND Recensisce.prodotto = \$prodottoId) context RecensisceService::allElements(string \$ordinamento) post: Return collection(Prodotto).
Invarianti	

Nome Classe	CarrelloService
Descrizione	Classe di business logic che gestisce le operazioni sul Carrello.
Pre-Condizione	context CarrelloService::aggiungiAlCarrello(Prodotto \$prodotto) pre: \$prodotto != null. context CarrelloService::svuotaCarrello() pre: nessuna pre-condizione. context CarrelloService::getProdottiCarrello() pre: nessuna pre-condizione. context CarrelloService:rimuoviDalCarrello(int \$codiceProdotto) pre: \$codiceProdotto != null
Post-Condizione	context CarrelloService::aggiungiAlCarrello(Prodotto \$prodotto) post: Return true se il prodotto era già presente nella collection 'Carrello' in sessione OR Return false se il prodotto non era presente e viene aggiunto alla collection 'Carrello' in sessione. context CarrelloService::svuotaCarrello() post: La collection 'Carrello' viene rimossa dalla sessione se esiste. context CarrelloService::getProdottiCarrello() post: (Return \$result AND \$result = collection(Prodotto) AND Per ogni Prodotto p in \$result vale: p è presente nella collection 'Carrello' in sessione se esiste) OR (Return collection()) context CarrelloService:rimuoviDalCarrello(int \$codiceProdotto) post: Per ogni Prodotto p nella collection 'Carrello' in sessione vale: p.codice_prodotto <> \$codiceProdotto AND ((Non esistono più Prodotti nella collection 'Carrello' AND la chiave 'Carrello' viene rimossa dalla sessione) OR (La collection 'Carrello' contiene tutti i Prodotti precedenti tranne quello con codice_prodotto = \$codiceProdotto))

Invarianti	
------------	--

Nome Classe	CategoriaService
Descrizione	Classe di business logic che gestisce le categorie dei prodotti.
Pre-Condizione	context CategoriaService::loadCategorie() pre: Nessuna pre-condizione. context CategoriaService::allElements(string \$ordinamento) pre: \$ordinamento != null AND \$ordinamento != "" AND \$ordinamento deve rispettare il formato di ORDER BY di SQL ovvero: “parametro asc/desc” dove “parametro” deve corrispondere a un attributo della tabella su cui facciamo l’ordinamento.
Post-Condizione	context CategoriaService::loadCategorie() post: Return sequence(Categoria). context CategoriaService::allElements(string \$ordinamento) post: Return sequence(Categoria).
Invarianti	

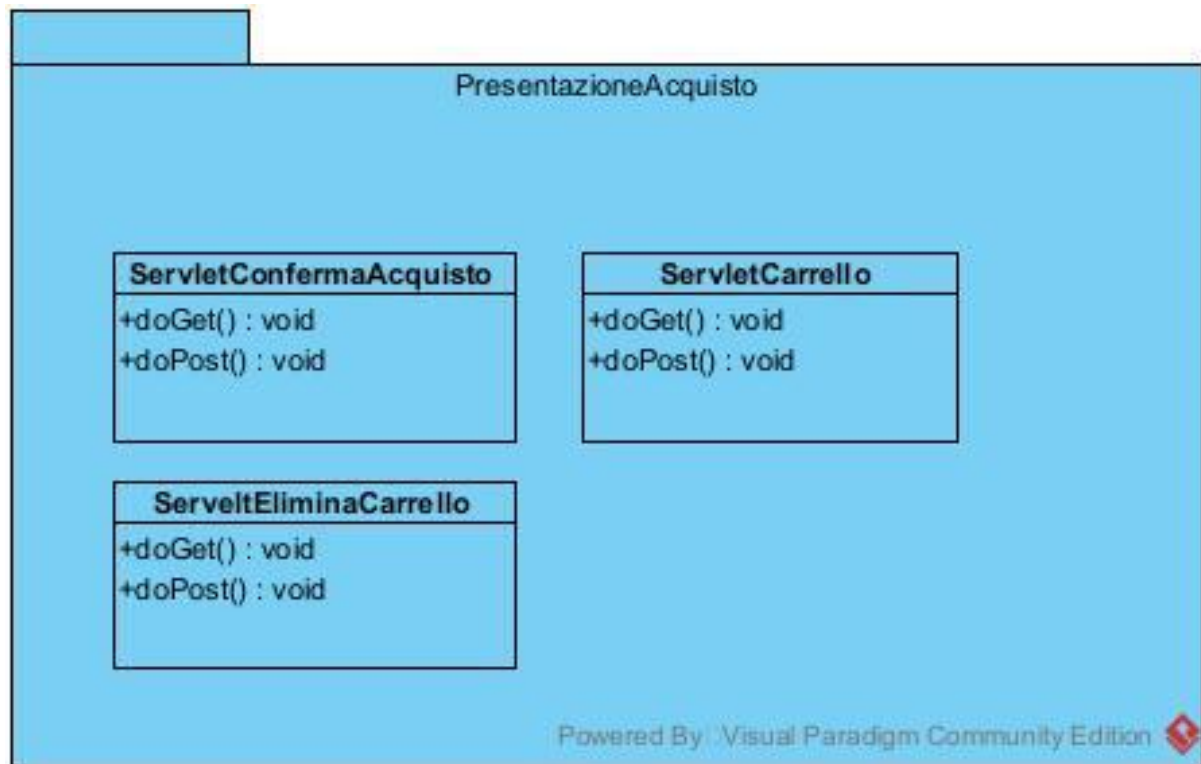
Nome Classe	FornitoreService
Descrizione	Classe di business logic che gestisce la fornitura dei prodotti.
Pre-Condizione	context FornitoreService::loadFornitori() pre: Nessuna pre-condizione. context FornitoreService::allElements(string \$ordinamento) pre: \$ordinamento != null AND \$ordinamento != "" AND \$ordinamento deve rispettare il formato di ORDER BY di SQL ovvero: “parametro asc/desc” dove “parametro” deve corrispondere a un attributo della tabella su cui facciamo l’ordinamento.
Post-Condizione	context FornitoreService::loadFornitori() post: Return sequence(Fornitore). context FornitoreService::allElements(string \$ordinamento) post: Return sequence(Fornitore).
Invarianti	

Nome Classe	SoftwareHouseService
Descrizione	Classe di business logic che gestisce le operazioni delle softwarehouse.
Pre-Condizione	context SoftwareHouseService::loadSoftwareHouse() pre: Nessuna pre-condizione. context SoftwareHouseService::allElements(string \$ordinamento) pre: \$ordinamento != null AND \$ordinamento != "" AND \$ordinamento deve rispettare il formato di ORDER BY di SQL ovvero: "parametro asc/desc" dove "parametro" deve corrispondere a un attributo della tabella su cui facciamo l'ordinamento.
Post-Condizione	context SoftwareHouseService::loadSoftwareHouse() post: Return sequence(SoftwareHouse). context SoftwareHouseService::allElements(string \$ordinamento) post: Return sequence(SoftwareHouse).
Invarianti	

Nome Classe	ParteDiService
Descrizione	Classe di business logic che gestisce le operazioni tra le categorie e i videogiochi.
Pre-Condizione	context ParteDiService::ricercaPerChiave(int \$id1, string \$id2) pre: \$id1>0 and \$id2 != null. context ParteDiService::allElements(string \$ordinamento) pre: \$ordinamento != null AND \$ordinamento != "" AND \$ordinamento deve rispettare il formato di ORDER BY di SQL ovvero: “parametro asc/desc” dove “parametro” deve corrispondere a un attributo della tabella su cui facciamo l’ordinamento. context ParteDiService::newInsert(ParteDi \$item) pre: \$item != null.
Post-Condizione	context ParteDiService::ricercaPerChiave(int \$id1, string \$id2) post: Return ParteDi OR null. context ParteDiService::allElements(string \$ordinamento) post: Return sequence(ParteDi). context ParteDiService::newInsert(ParteDi \$item) post: Viene inserita una tupla nel database nella tabella ParteDi con i valori che si trovano in \$item
Invarianti	

5.6. Fase 6

5.6.1. Package PresentazioneAcquisto

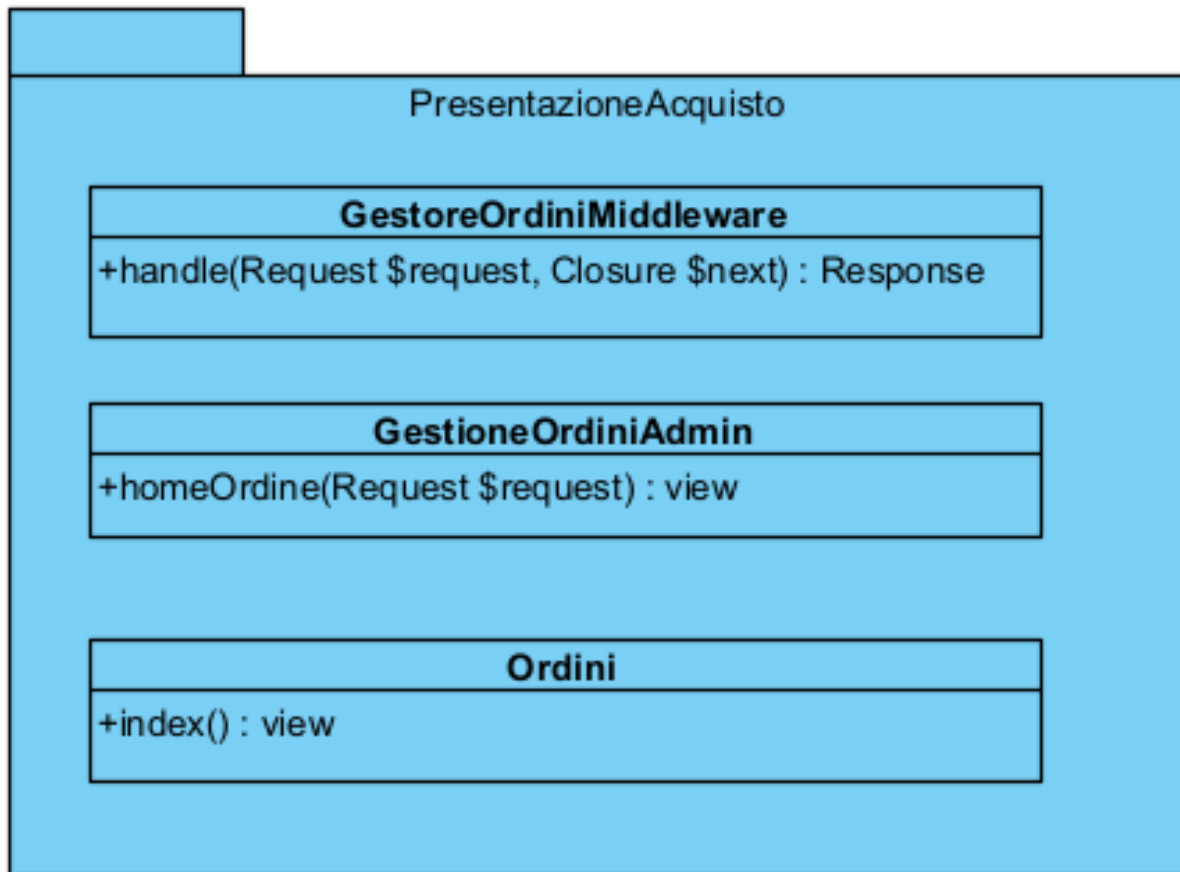


In questa fase della migrazione, come preventivato nei punti precedenti, si è intervenuti sia sul package **Acquisto** e sia sul package **PresentazioneAcquisto**. Partendo innanzitutto dal package **PresentazioneAcquisto**, è stata condotta un'analisi approfondita dei componenti presenti nel sistema legacy, costituiti dalle relative pagine JSP e Servlet (come riportato in figura). Tale package era infatti collocato all'interno del layer di presentazione e la sua analisi ha consentito di individuare con chiarezza le responsabilità dei singoli componenti e i diversi flussi applicativi gestiti dal sistema. Tra questi rientrano: la visualizzazione dello storico degli ordini, che in precedenza era gestita nei package **PresentazioneProdotto** e **Prodotto** e che, come spiegato nel punto precedente, è stata ricollocata in questa fase;

la visualizzazione della pagina del carrello, con la possibilità di eliminare prodotti dal carrello stesso o di confermare l'acquisto, previo superamento dei controlli previsti dal sistema.

Anche in questa fase non ci si è limitati a una semplice traduzione tecnologica verso il nuovo framework, ma si è proceduto a un vero e proprio processo di riprogettazione all'interno di Laravel, adottando un'architettura maggiormente coerente con i principi di separazione delle responsabilità e della business logic. Dall'analisi del sistema legacy è infatti emerso che i controller accentravano gran parte della logica applicativa, senza un'adeguata suddivisione dei compiti tra i diversi livelli dell'architettura. Per questo motivo, una volta ridefinito in modo strutturato il layer di presentazione, si è potuto intervenire anche sul package **Acquisto**, riorganizzandolo secondo una separazione più netta tra presentazione, controllo e dominio, in linea con le buone pratiche architetturali adottate nel nuovo contesto applicativo.

Il risultato finale:



Sostituzioni e Miglioramenti delle Servlet

Come anticipato nel punto precedente, le relative JSP e Servlet per la gestione degli ordini sono state ricollocate in questa fase della migrazione.

Entrando nel dettaglio dei controller, le due servlet:

- **ServletGestioneOrdiniAdmin.java**
- **ServletFormOrdiniAdmin.java**

sono state migrate in un unico controller Laravel denominato **GestioneOrdiniAdmin**. In particolare:

- **ServletGestioneOrdiniAdmin.java** trova corrispondenza nel metodo **homeOrdine**;
- **ServletFormOrdiniAdmin.java** è stata mappata nel metodo **formOrdiniAdmin**.

Già nella fase precedente, in concomitanza con la riorganizzazione della gestione dei magazzini, si era deciso di eliminare la business logic dalle servlet, delegandola a livelli più appropriati dell'architettura. Durante la migrazione del form, infatti, il codice risulta pulito e privo di logica applicativa, coerentemente con il principio di separazione delle responsabilità adottato nel nuovo contesto. Analogamente a quanto avvenuto nella fase 5, in cui era necessario controllare l'autorizzazione all'accesso ad alcune servlet (non l'autenticazione come amministratore, già gestita nella fase dedicata al profilo), anche in questo caso è stato introdotto un controllo basato sul ruolo "gestore ordini", distinto dal ruolo di "gestore prodotti". A tal fine è stato implementato un middleware dedicato, denominato **GestoreOrdiniMiddleware**, che consente di centralizzare e rimuovere il codice di controllo precedentemente presente nelle servlet.

È stata inoltre migrata **ServletOrdini.java** in una classe controller Laravel chiamata **Ordini**, nella quale la servlet originaria trova corrispondenza nel metodo **index**.

Per quanto riguarda invece la gestione dell'acquisto, nel sistema legacy erano presenti le seguenti servlet:

- **ServletCarrello.java**
- **ServletEliminaCarrello.java**
- **ServletConfermaAcquisto.java**

Tali componenti sono stati accorpati in un'unica classe Laravel denominata **Carrello**, all'interno della quale ciascuna servlet trova corrispondenza in uno specifico metodo:

- **carrello**
- **eliminaCarrello**

- **confermaAcquisto**

Questa scelta è coerente con l'approccio adottato in tutte le fasi della migrazione: Laravel consente infatti di raggruppare funzionalità omogenee all'interno dello stesso controller, differenziandole tramite metodi distinti, garantendo così maggiore coerenza strutturale, migliore organizzazione del codice e una più chiara separazione delle responsabilità.

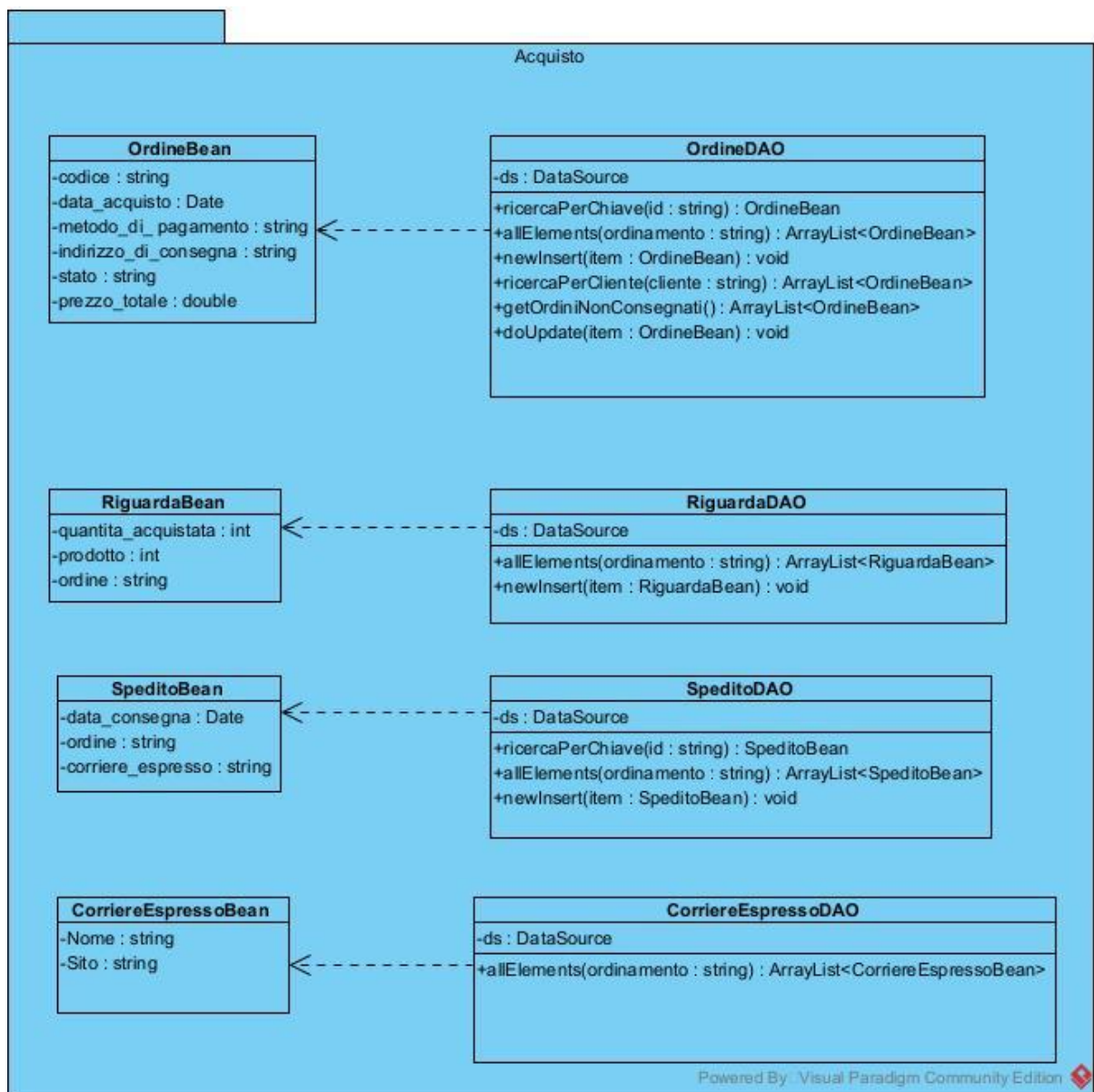
Sostituzioni e riutilizzo di JSP

Per quanto riguarda le JSP, come già anticipato nel punto precedente, la **paginaGestioneOrdini.jsp** è stata ricollocata dal modulo Prodotto al modulo Acquisto, infatti, in questa fase, si è proceduto alla migrazione delle seguenti pagine:

- **paginaGestioneOrdini.jsp**
- **paginaCarrello.jsp**
- **ordini.jsp**

La migrazione ha mantenuto la stessa denominazione dei file, modificandone unicamente l'estensione in **.blade.php**. In questo caso si è accorti che era possibile effettuare una migrazione sostanzialmente uno-a-uno, trasformando il codice JSP in sintassi Blade/PHP e adattando le direttive proprie dell'ambiente Java al nuovo framework. Pur cercando di preservare il più possibile la struttura originale, inclusi CSS e JavaScript, la migrazione non si è limitata a una conversione meccanica. Sono stati infatti apportati alcuni miglioramenti puntuali, sia a livello di organizzazione del markup sia in termini di responsività e ottimizzazione dell'interfaccia, al fine di rendere le pagine maggiormente coerenti con le buone pratiche attuali e con l'architettura complessiva del nuovo sistema. In questo modo, anche il layer di presentazione è stato riallineato non solo tecnologicamente, ma anche qualitativamente rispetto al sistema legacy, anche se di poco.

5.6.2. Package Acquisto



Dopo aver completato la migrazione del package

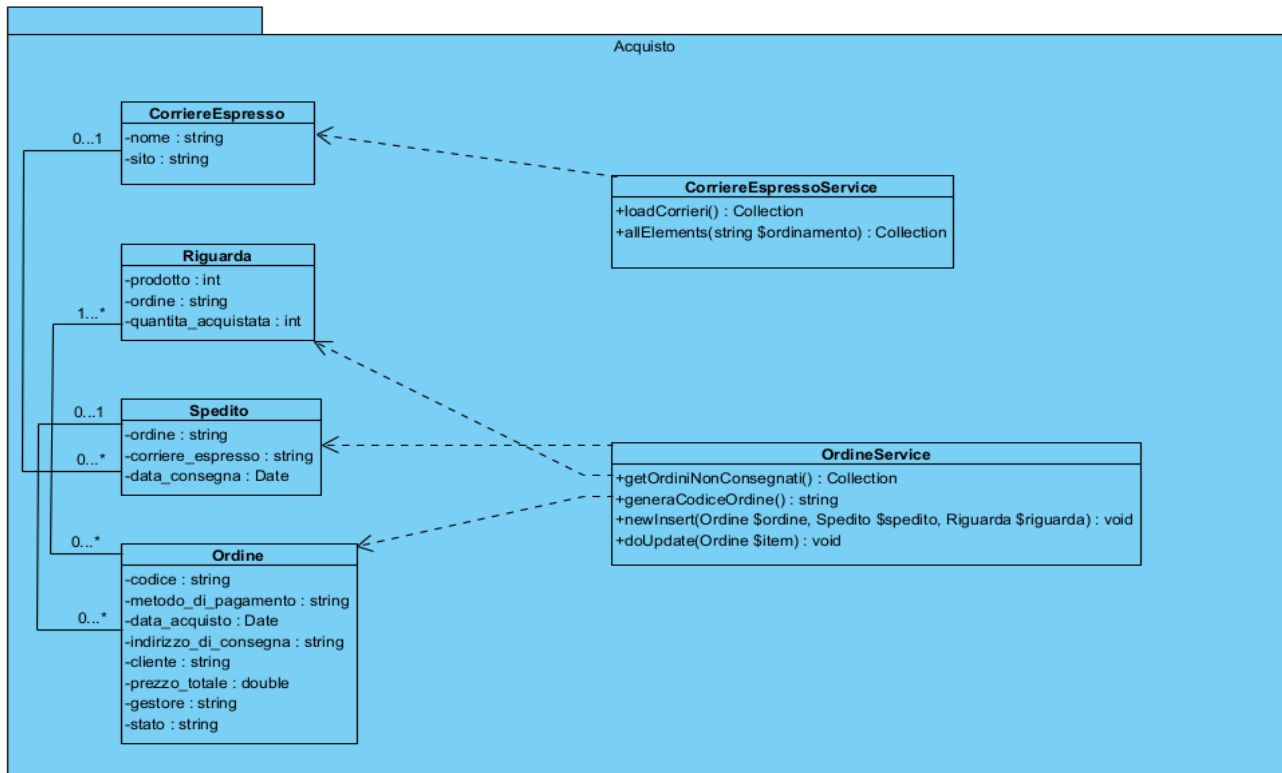
PresentazioneAcquisto, si è reso necessario intervenire sul package **Acquisto**, che, analogamente ai moduli trattati nelle fasi precedenti, rappresenta il livello deputato alla gestione della logica di dominio. Nel sistema legacy, tuttavia, tale package non racchiudeva una reale business logic, ma svolgeva prevalentemente un ruolo di supporto tecnico, strutturato secondo il classico schema DAO/DTO e orientato principalmente all'accesso ai dati. Coerentemente con l'approccio adottato nel nuovo contesto architetturale, tutti i bean sono stati trasformati in Model Laravel, arricchiti con le opportune relazioni ORM.

Questo ha consentito di rappresentare in modo più naturale e coerente le associazioni tra le entità del dominio (ordini, spedizioni, corrieri, ecc.), riducendo la necessità di query esplicite e favorendo una maggiore espressività del codice.

I DAO, invece, sono stati migrati in Service Laravel. In questa fase non ci si è limitati a una trasposizione uno-a-uno delle classi esistenti: per ciascun DAO è stata effettuata un'analisi puntuale delle responsabilità effettive, decidendo di volta in volta se mantenere, modificare o eliminare le singole funzioni. Un caso significativo è rappresentato da **CorriereEspressoDAO**, che è stato migrato in **CorriereEspressoService** mantenendo le funzionalità principali, ma arricchendole con logica di business e con l'introduzione di un meccanismo di cache a tempo. Tale scelta è coerente con quanto adottato nei moduli delle fasi precedenti: trattandosi di dati relativamente stabili e frequentemente consultati, si è preferito evitare accessi ripetuti al database, memorizzando temporaneamente le informazioni e invalidandole solo quando necessario. Diverso è il caso di **CorriereDAO** e **SpeditoDAO**. Dall'analisi del codice legacy è emerso che molte delle loro funzioni risultavano ridondanti o superflue rispetto alle potenzialità offerte dall'ORM. In particolare, operazioni come la ricercaPerChiave o la allElements, potevano essere ottenute direttamente tramite le relazioni tra i Model, senza la necessità di mantenere metodi dedicati. Per questo motivo, tali DAO non sono stati migrati integralmente. Per quanto riguarda **OrdineDAO**, la migrazione è stata invece più strutturata, infatti, la classe è stata trasferita in un Service dedicato, eliminando le funzioni ritenute inutili o prive di logica di dominio e riorganizzando quelle centrali. In particolare, il metodo newInsert è stato profondamente rivisto: non si tratta più di una semplice operazione di inserimento dati, ma di una vera e propria funzione di acquisto, che ingloba anche il comportamento precedentemente distribuito tra altri DAO (come quello relativo all'entità "Riguarda"). Allo stesso modo, le operazioni di aggiornamento (doUpdate) sono state ripensate in un'ottica maggiormente transazionale e coerente con la logica di business. Le modifiche non si limitano quindi a salvare dati, ma includono controlli, gestione dello stato dell'ordine e coordinamento tra entità correlate, garantendo maggiore coerenza e

integrità del dominio applicativo. Nel complesso, il package Acquisto non si configura più come un semplice livello di accesso ai dati.

Il risultato finale:



Nuove Class Interfaces

Nome Classe	CorriereEspressoService
Descrizione	Classe di business logic che gestisce le operazioni del corriere espresso.
Pre-Condizione	context CorriereEspressoService::loadCorrieri() pre: Nessuna pre-condizione. context CorriereEspressoService::allElements(string \$ordinamento) pre: \$ordinamento != null AND \$ordinamento != "" AND \$ordinamento deve rispettare il formato di ORDER BY di SQL ovvero: “parametro asc/desc” dove “parametro” deve corrispondere a un attributo della tabella su cui facciamo l’ordinamento.
Post-Condizione	context CorriereEspressoService::loadCorrieri() post: Return sequence(CorriereEspresso). context CorriereEspressoService::allElements(string \$ordinamento) post: Return sequence(CorriereEspresso).
Invarianti	

Nome Classe	OrdineService
Descrizione	Classe di business logic che gestisce le operazioni dell'Ordine.
Pre-Condizione	context OrdineService::getOrdiniNonConsegnati() pre: Nessuna pre-condizione. context OrdineService::generaCodiceOrdine() pre: Nessuna pre-condizione. context OrdineService::newInsert(Ordine \$ordine, Collection \$riguardaList) pre: \$ordine != null AND \$riguardaList != null AND \$riguardaList->notEmpty() AND Per ogni elemento r in \$riguardaList vale: r è istanza di Riguarda. context OrdineService::doUpdate(Ordine \$item) pre: \$item != null.
Post-Condizione	context OrdineService::getOrdiniNonConsegnati() post: \$result = sequence(Ordine) AND \$result->forAll(o o.gestore = null) AND Return \$result. context OrdineService::generaCodiceOrdine() post: \$result.matches('[0-9]+') AND \$result non esiste in alcuna tupla della tabella Ordine AND Return \$result context OrdineService::newInsert(Ordine \$ordine, Spedito \$spedito, Riguarda \$riguarda) post: (Ordine.allInstances()->exists(o o.codice = \$ordine.codice) AND Per ogni Riguarda r in \$riguardaList vale: Riguarda.allInstances()->exists(x x = r AND x.ordine = \$ordine.codice)) OR (NOT Ordine.allInstances()->exists(o o.codice = \$ordine.codice) AND Per ogni Riguarda r in \$riguardaList vale: NOT Riguarda.allInstances()->exists(x x = r)) context OrdineService::doUpdate(Ordine \$ordine, Spedito

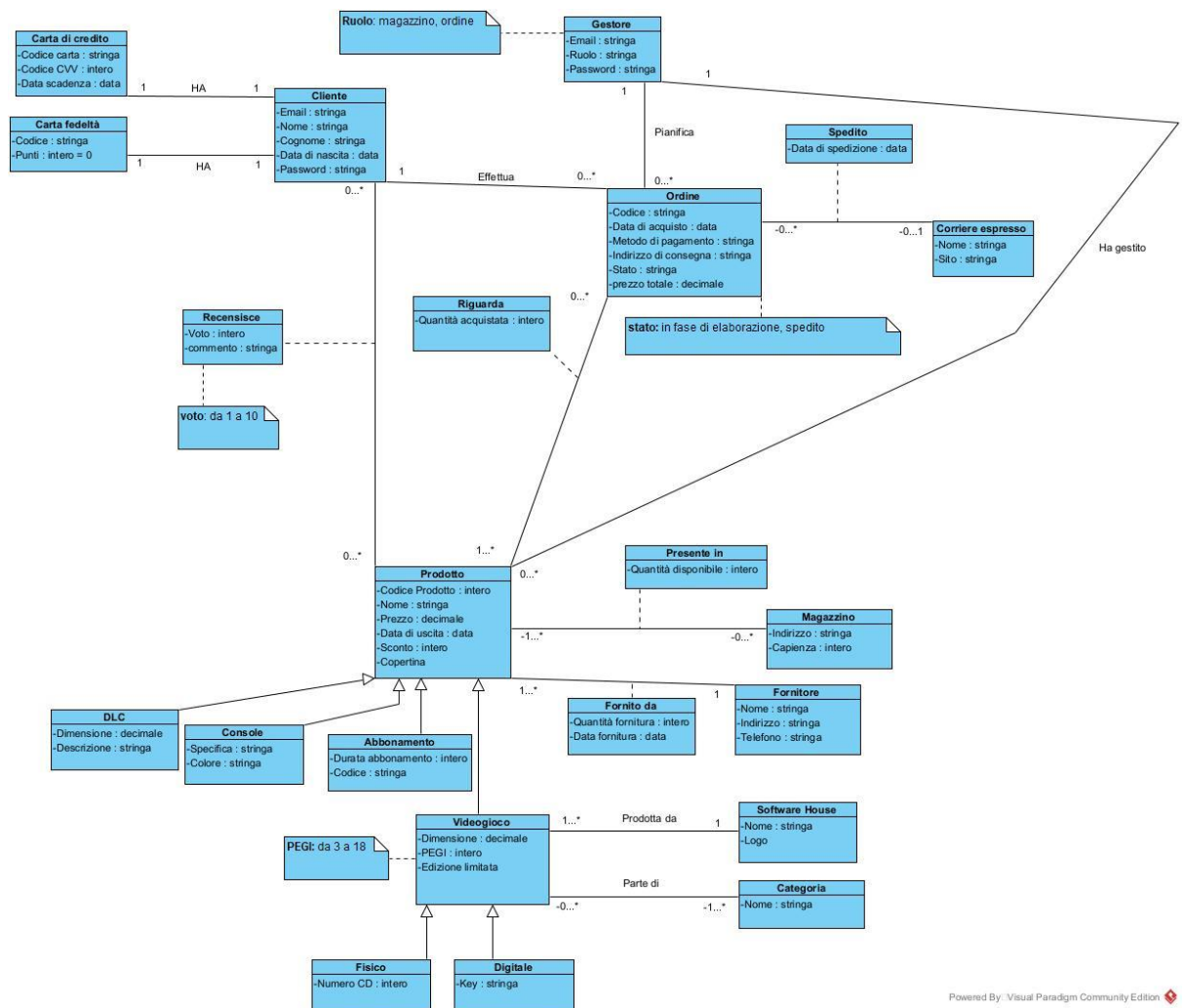
	\$spedito) post: Aggiorna la tupla nel database nella tabella Ordine corrispondente al codice interno al parametro ordine AND Inserisce la spedizione nella tabella Spedito con \$spedito.ordine = \$ordine.codice.
Invarianti	

5.7. Fase 7

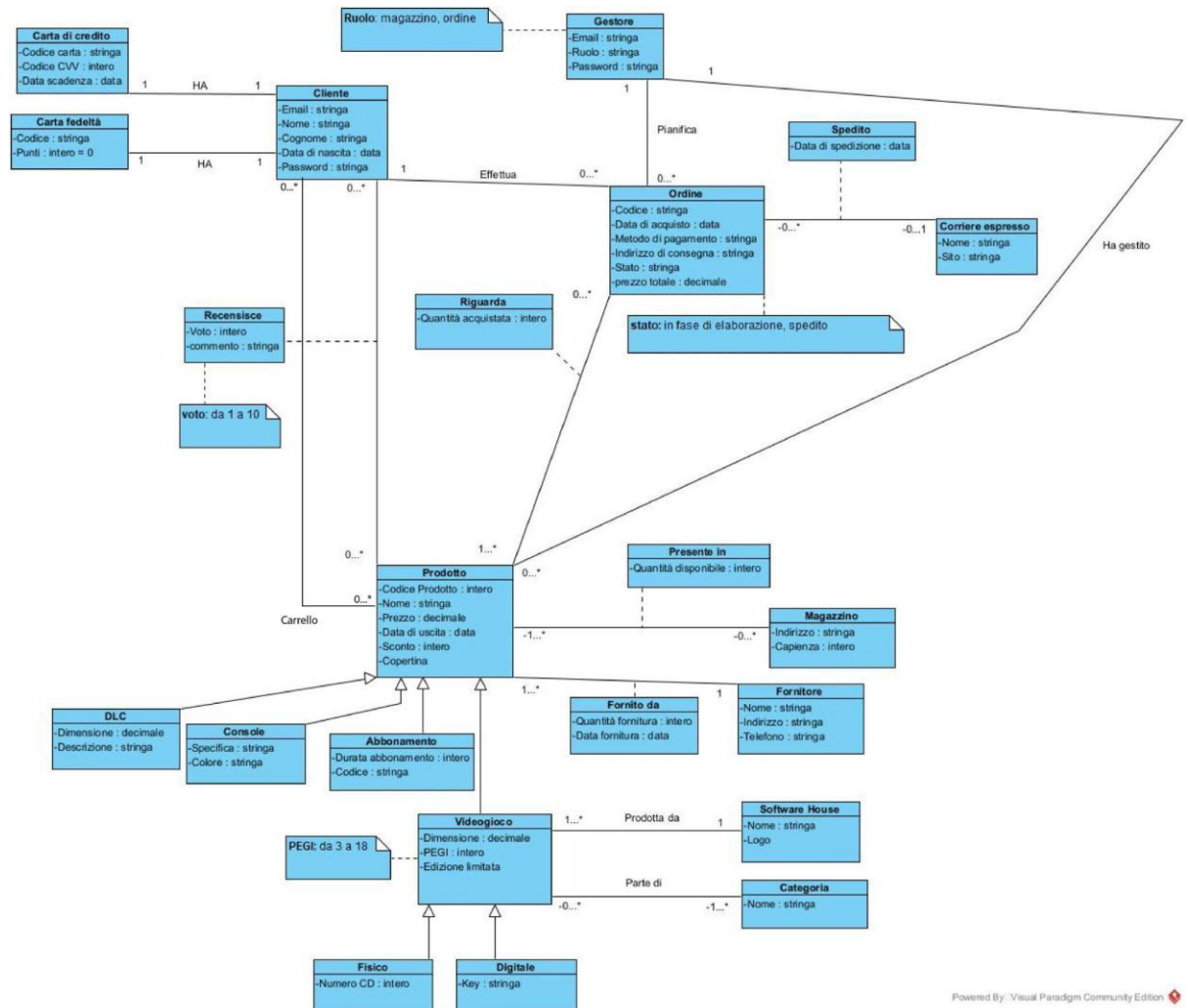
La relativa Fase7 riguarda completamente i test di sistema descritti nel documento di Master_Test_plan.

5.8. Class Diagram

Nonostante la migrazione fosse completata con successo, è stato successivamente corretto un piccolo errore strutturale nel Class Diagram, relativo alla mancata rappresentazione del carrello. Si tratta di un dettaglio minore che non ha avuto alcun impatto sul funzionamento del sistema.



Class Digram Prima



Class Diagram Dopo